Shark: Actively Secure Inference using Function Secret Sharing

Kanav Gupta* *University of Maryland*
Nishanth Chandran *Microsoft Research*
Divya Gupta *Microsoft Research*
Jonathan Katz *Google*
Rahul Sharma *Microsoft Research*

***Abstract*—We consider the problem of actively secure two-party machine-learning inference in the preprocessing model, where the parties obtain (input-independent) correlated randomness in an offline phase that they can then use to run an efficient protocol in the (input-dependent) online phase. In this setting, the state-of-the-art is the work of Escudero et al. (Crypto 2020); unfortunately, that protocol requires a large amount of correlated randomness, extensive communication, and many rounds of interaction, which leads to poor performance.**

In this work, we show protocols for this setting based on function secret sharing (FSS) that beat the state-of-the-art in all parameters: they use less correlated randomness and fewer rounds, and require lower communication and computation. We achieve this in part by allowing for a mix of boolean and arithmetic values in FSS-based protocols (something not done in prior work), as well as by relying on "interactive FSS," a generalization of FSS we introduce. To demonstrate the effectiveness of our approach we build SHARK—the first FSS-based system for actively secure inference—which outperforms the state-of-the-art by up to 2300×.

1. Introduction

In the problem of secure inference, two parties P_0 and P_1 interact to evaluate a machine-learning (ML) model held by P_0 on an input held by P_1 such that only the final result is revealed to (one or both of) the parties. This problem can be solved using generic secure two-party computation (2PC) [32], [69], and a significant body of work [31], [37], [53], [58] has developed efficient protocols tailored to this problem (see [49] for a survey).

To improve efficiency, both in the context of general 2PC as well as ML inference specifically, many works [22], [29], [34], [38], [42], [51], [53], [68] have considered protocols in the so-called *preprocessing model*. In that setting, P_0 and P_1 obtain input-independent correlated randomness during an offline phase, and can then use that correlated randomness during an input-dependent online phase. However, almost all these works achieve only *semi-honest security*, where the parties are assumed to follow the protocol honestly, and are not *actively secure*, i.e., they provide no security guarantees if either party is *malicious* and deviates from the protocol.

One exception to this is the work of Escudero et al. [29] (building on [22]), which is based on secret sharing and implemented in the MP-SPDZ framework [40]. While that work focuses on generic 2PC, it also considers ML inference as a special case and is the state-of-the-art for that problem. Unfortunately, the work of Escudero et al. suffers from significant overhead, both in terms of communication as well as rounds of interaction. (See Table 1.) As an example, their sub-protocol for computing $\mathsf{ReLU}(x) = \mathsf{max}(x, 0)$ (a function used heavily for ML inference) on n-bit inputs requires $\Theta(\log n)$ rounds of interaction and $7n + 3s$ bits of communication, where $s \approx 64$ is related to the statistical security parameter. As another example, their protocol for piece-wise polynomials (aka splines) of degree d with m intervals uses $\Theta(d + \log n)$ rounds and $\Theta((m + d)(n + s))$ bits of communication. It is thus worthwhile to explore other approaches for secure inference in this setting.

Another paradigm for 2PC in the preprocessing model is based on *function secret sharing* (FSS) [6], [7], [8], [9]. This technique enables round- and communication-efficient protocols for many functions, and has been used to construct state-of-the-art semi-honest ML-inference protocols [34], [35], [38], [62]. However, existing FSS-based protocols require a large amount of correlated randomness and have high computational overhead compared to protocols based on secret sharing. Indeed, all existing implementations of FSS-based protocols [34], [35], [38], [62] only consider semi-honest security. While there is work [6] showing a theoretical approach for adapting semi-honest FSS-based protocols to provide malicious security, that work has several limitations. First, it is not applicable to arbitrary FSS-based protocols, in particular to protocols that require bitwise operations (e.g., for computing truncations) or operations involving boolean values (e.g., for comparisons) more generally. Second, and more problematic for efficiency, that approach requires underlying FSS schemes to be run on inputs of length $n + s$, as opposed to inputs of length n as in the semi-honest case. Finally, no implementations of the approach exist, making its practical efficiency unclear.

1.1. Our Contributions

We present SHARK, the first implementation of an FSS-based protocol for actively secure inference, which is up to three orders of magnitude faster than existing state-of-the-art protocols for actively secure inference in the preprocessing model [29]. It is also more efficient than the existing (unimplemented) approach for designing actively secure FSS-based protocols [6]. At its core, SHARK achieves

*. Part of this work was done while at Microsoft Research.

these efficiency improvements by exploiting a different way of lifting FSS-based protocols to the malicious setting, by considering a generalization of FSS called "interactive FSS," and by developing efficient techniques for computing on a mix of boolean and arithmetic values; see Sections 1.2 and 3.2 for further details. These ideas allow us to develop new FSS-based subprotocols for functionalities commonly used for ML inference, such as ReLU, arithmetic right shift (ARS), ReluARS (that fuses ReLU and ARS), and piece-wise polynomials (splines), that improve significantly on prior work [6], [29] (cf. Table 1):

- Compared to the state-of-the-art in actively secure inference [29], our protocols use up to $15\times$ less communication and have significantly fewer rounds. While FSS-based protocols are known to reduce communication and round complexity at the cost of more preprocessing material, our protocols—somewhat surprisingly—actually use less preprocessing material. We thus *improve on the state-of-the-art in all relevant parameters*. (See also Table 2.)
- Compared to state-of-the-art actively secure FSS-based protocols [6], our protocols use up to $70\times$ less preprocessing material, and $58\times$ less computation (measured in terms of AES evaluations) while keeping the online communication and round complexity nearly the same (at most one extra round). We also provide the first actively secure FSS-based protocol for computing reciprocals, crucial for evaluating the softmax function in transformers.

We run experiments for end-to-end secure inference using benchmarks such as HiNet, AlexNet, and VGG16 (cf. Table 3), and find that SHARK is up to $2300\times$ faster (depending on the setting and benchmark considered) than the work of Escudero et al. [29]. SHARK is also sufficiently expressive to support the *first* actively secure inference of transformers (see Section 6.2).

		AES calls	Comm. (bits)	Preprocessing size (bits)	Rounds
ReLU Section 5.1	SHARK	$2(n-1)$ (126)	$\tilde{n}+1$ (129)	$n(\lambda+\kappa)$ (11,879)	2
	[6]	$12\tilde{n}$ (1536)	$\tilde{n}$ (128)	$\tilde{n}(\lambda+4\tilde{n})$ (83,838)	1
	[29]	—	$7n+3s$ (621)	$7n\kappa$ (18,459)	$\mathcal{O}(\log n)$
ARS Section 5.2	SHARK	$2(n+f)$ (160)	$\tilde{n}+2$ (130)	$(n+f)(\lambda+\kappa)$ (14,960)	2
	[6]	$4\tilde{n}$ (512)	$\tilde{n}$ (128)	$\tilde{n}(\lambda+2\tilde{n})$ (49,790)	1
	[29]	—	$6n+2s$ (482)	$7n\kappa$ (17,506)	$\mathcal{O}(\log n)$
ReluARS Section 5.3	SHARK	$4n+2f$ (288)	$\tilde{n}+3$ (131)	$(n+f)(\lambda+\kappa)$ (16,537)	2
	[6]	$16\tilde{n}$ (2048)	$2\tilde{n}$ (256)	$\tilde{n}(2\lambda+6\tilde{n})$ (133,628)	2
	[29]	—	$13n+5s$ (1,103)	$14n\kappa$ (35,965)	$\mathcal{O}(\log n)$
Spline Section 5.4	SHARK	$4mn$ (4,864)	$(d+2)\tilde{n}$ (512)	$n(\lambda+2\tilde{n})$ $+6d\tilde{n}$ (27,262)	2
	[6]	$2m^2(d+1)\tilde{n}$ (282,112)	$\tilde{n}$ (128)	$2\tilde{n}m^2(d+1)$ (1,928,318)	1
	[29]	—	$m(5n+s)$ $+2d\tilde{n}$ (7,703)	$7mn\kappa+12d\tilde{n}$ (338,433)	$\mathcal{O}(\log n$ $+d)$

TABLE 1: Comparing SHARK to prior work, asymptotically and concretely (in parentheses). All numbers include the final reconstruction step. Here, λ, κ, s, n, f denote the computational security parameter (set to 126 for the concrete comparison), statistical security parameter (40), $\mathsf{SPD}\mathbb{Z}_{2^k}$ slack (64), input length (64), and truncation output length (16), respectively. For a spline, m is the number of intervals (19) and d is the polynomial degree (2). We let $\tilde{n} = n + s$.

1.2. Overview of Our Techniques

In what follows, we assume the underlying computation is done on n-bit values and let $N = 2^n$. At a high level, semi-honest FSS-based 2PC protocols operate by having the parties hold a masked value $\hat{x} = x + \mathsf{r}_{\mathsf{in}} \in \mathbb{Z}_N$ of each intermediate wire value $x \in \mathbb{Z}_N$, where $\mathsf{r}_{\mathsf{in}} \in \mathbb{Z}_N$ is a random mask that is chosen during preprocessing. The parties then use FSS to compute shares of $\hat{y} = g(\hat{x}) \stackrel{\text{def}}{=} f(\hat{x} - \mathsf{r}_{\mathsf{in}}) + \mathsf{r}_{\mathsf{out}}$ for some desired function f and output mask $\mathsf{r}_{\mathsf{out}}$; by exchanging their shares, both parties learn $\hat{y}$ and can continue evaluation. To provide security against malicious behavior [6], the protocol can be modified so that parties obtain masked, *authenticated* values, where the authentication tags are checked at the end of the computation. This approach requires masked values to now be $(n+s)$-bits long (where s is related to a statistical security parameter) and FSS schemes to be applied to $(n+s)$-bit inputs, resulting in increased computation and more preprocessing material.

We apply several techniques to improve the efficiency of FSS-based protocols in the malicious setting. For starters, we return to the original invariant of having the parties hold n-bit masked values. Those values are derived from authenticated, masked values (note the change in order compared to [6]) that can still be checked at the end of the computation to ensure integrity. While seemingly a minor change, this allows underlying FSS computations to now be done on n-bit values, resulting in improved efficiency and reduced preprocessing material. Furthermore, inspired by prior work in the semi-honest setting [34], [38], we show how to construct actively secure FSS-based protocols that support a mix of arithmetic (i.e., n-bit) and boolean (i.e., 1-bit) values (something not directly supported by $\mathsf{SPD}\mathbb{Z}_{2^k}$ [19]) by relying on the boolean authenticated secret-sharing scheme of Frederiksen et al. [30]. Along with a generalization of FSS we call interactive FSS, this allows us to design FSS-based protocols for functions commonly used in ML inference with much better efficiency than prior work, as shown in Table 1. It also allows us to design protocols for functionalities not handled in prior work, such as the reciprocal function.

1.3. Other Related Work

Secure machine learning. The problem of secure inference was perhaps first introduced by Lindell and Pinkas [47]. Following this, a long line of work has considered this problem in the two-party semi-honest setting for convolutional neural networks [1], [5], [11], [16], [31], [35], [37], [38], [39], [42], [51], [53], [57], [58], [62], [68]. Some works have considered actively secure inference in the honest-majority setting (with 3 or more parties) [12], [17], [20], [43], [52], [56], [66]. Only a few works have considered actively secure inference in the more challenging 2-party case [21], [22], [29], [70]. Keller and Sun [41] use the protocols of Escudero et al. [29] for secure training. Recent works build on the above to provide protocols for secure inference of transformer models (with and without preprocessing) in the 2- and 3-party settings, but are restricted to the semi-honest model [2], [27], [34], [36], [45], [55], [71].

One-sided security. Muse [44], SIMC [15], and Fusion [26] consider protocols with one-sided security, i.e., they offer security against semi-honest corruption of one party or malicious corruption of the other. DETI [10] considers a similar security guarantee in the three-party case. SHARK is secure against malicious corruption of either party.

Generic secure computation. SPDZ [23] and subsequent optimizations thereof constitute the state-of-the-art for actively secure dishonest-majority secure computation in the preprocessing model. The original SPDZ protocol [23] used authenticated secret sharing over an arbitrary finite field. Cramer et al. [19] showed how to extend those ideas to work over rings, e.g., $\mathbb{Z}_{2^n}$. Authenticated secret sharing has also been used for boolean circuits [4], and a more efficient scheme was provided by Frederiksen et al. [30]. Subsequent works have devised techniques that allow for a hybrid of ring and boolean values [29], [60].

FSS-based protocols. Function secret sharing, introduced by Boyle et al. [7], [8], has been used to construct 2PC protocols in the preprocessing model [9]. A series of works [6], [34], [35], [38], [62], [64], [65] improved the efficiency of such protocols; some of those works also provide custom protocols for secure ML training and inference. Pika [65] shows an FSS-based protocol for actively secure inference in the 3-party, honest-majority setting.

1.4. Organization of the Paper

Section 2 introduces preliminaries, including background on authenticated secret sharing and function secret sharing. It also introduces a defintion extending the notion of FSS to an interactive setting. In Section 3 we discuss the general design of our overall protocol and how it differs from prior work. After reviewing basic FSS schemes in Section 4, we describe our (interactive) FSS schemes for non-linear functions (ReLU, ARS, and ReluARS, splines, and reciprocal) in Section 5. We introduce and evaluate SHARK, which incorporates all those schemes, in Section 6.

2. Preliminaries

We denote the computational and statistical security parameters by λ and κ, respectively, and implicitly assume they are provided as input to algorithms as needed. We let $[m] = \{0, 1, 2 \dots m-1\}$. Letting $N = 2^n$, we write $\mathbb{Z}_N$ for the ring of integers modulo N, and work with the unsigned integers $\{0, \dots N-1\}$ as representatives; in particular, this means truncation is well-defined and we can meaningfully compare elements of $\mathbb{Z}_N$. We sometimes also view elements in $\mathbb{Z}_N$ as signed integers using two's complement representation; note that addition/multiplication of signed numbers in this representation is equivalent to addition/multiplication modulo N (assuming no overflow). For $b \in \{0,1\}$, we write $\mathsf{B2A}_n(b)$ ("binary-to-arithmetic") to denote the natural lifting of b to $\mathbb{Z}_N$.

We use fixed-point representation to encode real numbers as elements of $\mathbb{Z}_N$. For precision f, a real number r is represented as $\lfloor r \cdot 2^f \rfloor \in \mathbb{Z}_N$. Conversely, in this case $x \in \mathbb{Z}_N$ represents the real number $x/2^f$.

For a predicate p, the indicator function $\mathbb{1}\{p\}$ is equal to 1 if p is true and 0 otherwise. We write arrays using boldface letters, and denote the ith element of an array $\mathbf{T}$ by $\mathbf{T}[i]$. We let $\mathbb{G}$ denote a finite group (written additively).

2.1. Problem Statement

We consider two parties P_0, P_1, with one holding model weights w for a public model architecture $\mathcal{M}$ and the other holding an input x; the parties want to compute $\mathcal{M}(w, x)$ without leaking anything additional about their inputs. While not essential for our results, we assume model architectures in which linear layers (involving fixed-point matrix multiplications) alternate with non-linear layers that may involve truncations. The non-linear operations can be simple (e.g., ReLU) or complex (e.g., sigmoid, tanh). We fuse truncation and simple non-linear functions, and approximate complex non-linear functions by splines. Transformers additionally require reciprocal operations.

As in prior work [19], [23], [30], [34], [38], [68], we assume a trusted dealer who provides input-independent preprocessing material to P_0, P_1 that allows them to perform the desired secure computation. We model the dealer as an ideal functionality $\mathcal{F}_{\mathsf{dealer}}$. The dealer can be instantiated using one of several techniques, e.g., specialized secure-computation protocols run as part of a preprocessing phase [19], [23], [30], an external trusted party [6], [9], [34], [38], [68], or trusted hardware. We assume a PPT adversary $\mathcal{A}$ who can statically corrupt one of the two parties and deviate arbitrarily from the prescribed protocol. Our proofs can be viewed as demonstrating universal composability [13] of our protocols in the $\mathcal{F}_{\mathsf{dealer}}$-hybrid model.

2.2. Authenticated Secret Sharing

Two-party secret sharing allows a dealer to split a value x in a finite group $\mathbb{G}$ into two shares x_0, x_1 such that neither share reveals anything about x, yet the shares

jointly allow for reconstruction of x; each share can then be given to one of the parties. In standard additive secret sharing one share is uniform in $\mathbb{G}$ and the other is chosen so the two shares sum to the desired value x. While this ensures secrecy, it does not allow either party to detect modification of a share by the other party. Prior work shows how to achieve such robustness via *authenticated secret sharing* [4], [19], [23], [30], two forms of which we rely on here: *arithmetic* authenticated secret sharing when $x \in \mathbb{Z}_N$, and *boolean* authenticated secret sharing when $x \in \{0,1\}$.

Arithmetic authenticated secret sharing. In the arithmetic case, we use the (two-party) authenticated secret-sharing scheme from $\mathsf{SPD}\mathbb{Z}_{2^k}$ [19]. Say a dealer wants to share values in $\mathbb{Z}_N$ for $N = 2^n$. Let s satisfy $s - \log(s) \geq \kappa$, and let $\widetilde{N} = 2^{n+s}$. The dealer chooses uniform values $\Delta_0^{\mathsf{A}}, \Delta_1^{\mathsf{A}} \in \mathbb{Z}_{2^s}$ that define a global MAC key $\Delta^{\mathsf{A}} = \Delta_0^{\mathsf{A}} + \Delta_1^{\mathsf{A}} \bmod \widetilde{N}$, and gives Δ_b^{A} to P_b. To generate a fresh sharing of a value $x \in \mathbb{Z}_N$, the dealer chooses uniform $x_0, x_1, \mathsf{t}_0, \mathsf{t}_1 \in \mathbb{Z}_{\widetilde{N}}$ satisfying

$$x = x_0 + x_1 \bmod N$$
$$\Delta^{\mathsf{A}} \cdot (x_0 + x_1) = \mathsf{t}_0 + \mathsf{t}_1 \bmod \widetilde{N},$$

and gives (x_b, t_b) to P_b. We denote this process by $((x_0, \mathsf{t}_0), (x_1, \mathsf{t}_1)) \leftarrow \mathsf{auth\text{-}ashare}(x)$, and further denote any tuple $((x_0, \mathsf{t}_0), (x_1, \mathsf{t}_1))$ satisfying the above equations (regardless of their distribution) by $[\![x]\!]$ with $[\![x]\!]_b = (x_b, \mathsf{t}_b)$. Homomorphic addition of shared values, or multiplication/addition of a shared value by a public constant $c \in \mathbb{Z}_N$, can be done using local computation; we denote these operations by $[\![x]\!] + [\![y]\!]$, $c \cdot [\![x]\!]$, and $c + [\![x]\!]$, respectively.

(Overloading notation slightly, for $x \in \mathbb{Z}_{\widetilde{N}}$, we also write $\mathsf{auth\text{-}ashare}(x)$ to denote choosing $x_0, x_1, \mathsf{t}_0, \mathsf{t}_1$ as above, and write $[\![x]\!]$ for shares satisfying the above equations. Although only $x \bmod N$ will be authenticated, this is acceptable for our purposes since we use this only when $x \in \mathbb{Z}_{\widetilde{N}}$ is a random mask.)

To reconstruct a shared value[1] $[\![x]\!] = ((x_0, \mathsf{t}_0), (x_1, \mathsf{t}_1))$, the parties exchange x_0, x_1 and set $x := x_0 + x_1 \bmod N$. They then verify that $(x_0 + x_1) \cdot \Delta^{\mathsf{A}} = \mathsf{t}_0 + \mathsf{t}_1 \bmod \widetilde{N}$. This is equivalent to checking that the values $z_0 = \mathsf{t}_0 - (x_0 + x_1) \cdot \Delta_0^{\mathsf{A}}$ and $z_1 = \mathsf{t}_1 - (x_0 + x_1) \cdot \Delta_1^{\mathsf{A}}$ sum to 0 in $\mathbb{Z}_{\widetilde{N}}$, which can be done by having the parties commit to z_0, z_1, respectively, and then open their commitments to verify the sum. Note that even though the reconstructed value x lies in $\mathbb{Z}_N$, it is crucial to exchange the $(n+s)$-bit values $x_0, x_1 \in \mathbb{Z}_{\widetilde{N}}$ since they are required for verification. Hence, reconstruction of a single n-bit value requires per-party communication of $2(n+s) + C$ bits, where C is the size of the commitments/decommitments to z_0, z_1.

Batch checking can be used to amortize the cost of verification over multiple reconstructed values [19], [23]. Say the parties want to reconstruct $[\![x^{(1)}]\!], \ldots, [\![x^{(\ell)}]\!]$. Each time shares $x_0^{(i)}, x_1^{(i)}$ are revealed, party P_b locally computes and stores $z_b^{(i)} := \mathsf{t}_b^{(i)} - (x_0^{(i)} + x_1^{(i)}) \cdot \Delta_b^{\mathsf{A}} \in \mathbb{Z}_{\widetilde{N}}$. Once all values have been reconstructed, the parties jointly sample (pseudo)random values $\chi^{(1)}, \ldots, \chi^{(\ell)} \in \mathbb{Z}_{2^s}$, after which P_b computes $z_b = \sum_i \chi^{(i)} \cdot z_b^{(i)} \bmod \widetilde{N}$. The parties then commit to z_0, z_1, respectively, and then open their commitments to check that they sum to 0 modulo $\widetilde{N}$ as before. The per-party amortized communication per reconstructed value is then only $n+s$ bits. We write $x \leftarrow \mathsf{auth\text{-}reconstruct}([\![x]\!])$ to denote batch reconstruction of $[\![x]\!]$, meaning that parties exchange their shares and locally compute and store the z-value as described above, with a final check of all reconstructed values defered to the end of the overall protocol.

Boolean authenticated secret sharing. For authenticated secret sharing of a boolean value, we use *FKOS sharing* as introduced by Frederiksen et al. [30]. Here, parties P_0 and P_1 are given uniform values $\Delta_0^{\mathsf{B}}, \Delta_1^{\mathsf{B}} \in \{0,1\}^\kappa$, respectively, that define a global MAC key $\Delta^{\mathsf{B}} = \Delta_0^{\mathsf{B}} \oplus \Delta_1^{\mathsf{B}} \in \{0,1\}^\kappa$. To share a value $x \in \{0,1\}$, the dealer chooses uniform $x_0, x_1 \in \{0,1\}$ and $\mathsf{t}_0, \mathsf{t}_1 \in \{0,1\}^\kappa$ such that

$$x = x_0 \oplus x_1 \text{ and } x \cdot \Delta^{\mathsf{B}} = \mathsf{t}_0 \oplus \mathsf{t}_1,$$

and gives (x_b, t_b) to P_b. We denote this process by $((x_0, \mathsf{t}_0), (x_1, \mathsf{t}_1)) \leftarrow \mathsf{auth\text{-}bshare}(x)$, and denote any shares $((x_0, \mathsf{t}_0), (x_1, \mathsf{t}_1))$ satisfying the above equations (regardless of their distribution) by $\langle x \rangle$ with $\langle x \rangle_b = (x_b, \mathsf{t}_b)$. Homomorphic XOR of shared values, or AND/XOR of a shared value with a public constant $c \in \{0,1\}$, can be done using local computation; we denote these by $\langle x \rangle \oplus \langle y \rangle$, $c \cdot \langle x \rangle$, and $c \oplus \langle x \rangle$, respectively.

Reconstruction with batch checking is done in a manner similar to that described above for the arithmetic case. To reconstruct $\langle x \rangle = ((x_0, \mathsf{t}_0), (x_1, \mathsf{t}_1))$, the parties exchange x_0, x_1 to learn $x := x_0 \oplus x_1$. Party P_b then stores $z_b := \mathsf{t}_b \oplus x \cdot \Delta_b^{\mathsf{B}}$, viewed as an element of the field $\mathbb{F}_{2^\kappa}$. Once all values have been reconstructed, the parties calculate a random linear combination of all stored z_b values and check equality (using commit-and-open). This way, the per-party amortized communication per reconstructed value is only 1 bit. We denote this process of reconstructing boolean shares by $x \leftarrow \mathsf{auth\text{-}reconstruct}(\langle x \rangle)$.

2.3. Function Secret Sharing (FSS)

We review the notion of (2-party) FSS schemes [7], [8].

Definition 1. *A (2-party) FSS scheme for function family $\mathcal{G} = \{g_p : \{0,1\}^* \to \mathbb{G}\}$ is a pair of efficient algorithms $(\mathsf{Gen}, \mathsf{Eval})$ such that*

- $\mathsf{Gen}(p)$ *is a randomized algorithm that outputs (k_0, k_1).*
- $\mathsf{Eval}(k_b, x)$ *takes as input a key k_b and $x \in \{0,1\}^*$, and outputs $y_b \in \mathbb{G}$.*

Correctness requires that for all p and $x \in \{0,1\}^*$, if $(k_0, k_1) \leftarrow \mathsf{Gen}(p)$ then $\mathsf{Eval}(k_0, x) + \mathsf{Eval}(k_1, x) = g_p(x)$.

For an FSS-scheme $\mathsf{F} = (\mathsf{Gen}, \mathsf{Eval})$, we use $\mathsf{keysize}(\mathsf{F})$ to denote $\max\{|k_0|, |k_1|\}$. Security requires that a single key does not reveal any information about p. Formally:

1. When $[\![x]\!]$ is not a fresh sharing of x, an additional re-randomization step is needed before reconsruction to prevent information leakage [19]. In our protocol, reconstructed values will already be suitably randomized so no explicit re-randomization is needed.

Definition 2. *An FSS scheme for $\mathcal{G}$ is secure if there is an efficient simulator $\mathcal{S}$ such that for every index p and $b \in \{0,1\}$, the following distributions are computationally indistinguishable:*

1) Run $(k_0, k_1) \leftarrow \mathsf{Gen}(p)$ and output k_b.
2) Output $\mathcal{S}(b)$.

2.4. Interactive FSS

We introduce a generalization of FSS schemes called *interactive FSS* (IFSS). An IFSS scheme also relies on a key-generation procedure, following which parties can compute shares of output given a common input. In contrast to FSS schemes, however, IFSS schemes allow interaction between the parties when computing their output shares. They also allow the parties to additionally output some auxiliary information at the end of the protocol.

Definition 3. *A (2-party) IFSS scheme for function family $\mathcal{G} = \{g_p : \{0,1\}^* \to \mathbb{G}\}$ and a randomized function AUX is a pair of efficient procedures (Gen, Π) such that*

- *$\mathsf{Gen}(p)$ is a randomized algorithm that outputs (k_0, k_1).*
- *$\Pi = (\Pi_0, \Pi_1)$ defines a two-party interactive protocol, where Π_b takes as input a key k_b and an input x, and outputs $y_b \in \mathbb{G}$ and $\mathsf{aux}_b \in \{0,1\}^*$.*

Correctness requires that for all p and $x \in \{0,1\}^*$, if we run $(k_0, k_1) \leftarrow \mathsf{Gen}(p)$ followed by

$$((y_0, \mathsf{aux}_0), (y_1, \mathsf{aux}_1)) \leftarrow \langle \Pi_0(k_0, x), \Pi_1(k_1, x) \rangle$$

then $y_0 + y_1 = g_p(x)$ and $(\mathsf{aux}_0, \mathsf{aux}_1)$ has the same distribution as $\mathsf{AUX}(p)$.

(While it is possible to allow AUX to also take x as input, this will not be needed in our constructions.) We say AUX is *efficiently simulatable* if (for $b \in \{0,1\}$) given p, aux_b it is possible to efficiently sample from the marginal distribution of aux_{1-b} conditioned on aux_b. That is, there is an efficient algorithm $\mathcal{S}_{\mathsf{AUX}}$ such that for any p and $b \in \{0,1\}$ the distributions $\mathsf{AUX}(p)$ and

$$\left\{ \begin{array}{c} (\mathsf{aux}_0^*, \mathsf{aux}_1^*) \leftarrow \mathsf{AUX}(p) \\ \mathsf{aux}_b := \mathsf{aux}_b^* \\ \mathsf{aux}_{1-b} \leftarrow \mathcal{S}_{\mathsf{AUX}}(b, p, \mathsf{aux}_b) \end{array} : (\mathsf{aux}_0, \mathsf{aux}_1) \right\}$$

are identical. We assume AUX is efficiently simulatable (as is the case in our constructions).

We define security in a way inspired by (but not equivalent to) standard definitions of secure two-party computation. As in standard definitions, we define real- and ideal-world executions and require them to be computationally indistinguishable. But in our setting, those executions are different from the standard ones. In particular, the specific function g_p being evaluated is unknown to the simulator in the ideal world, and we explicitly incorporate the keys generated at the outset of the execution. Details follow.

The *real-world* execution in the presence of a stateful adversary $\mathcal{A}$ corrupting P_b is defined as follows:

1) $\mathcal{A}$ outputs p.
2) Run $(k_0, k_1) \leftarrow \mathsf{Gen}(p)$, and give k_b to $\mathcal{A}$.
3) $\mathcal{A}$ outputs x.
4) Run $\Pi_{1-b}(k_{1-b}, x)$ with $\mathcal{A}$. Let $(y_{1-b}, \mathsf{aux}_{1-b})$ be the output of Π_{1-b} at the end of the execution.

We let $\text{REAL}_{\mathcal{A}}(1^\lambda)$ be the random variable corresponding to $(y_{1-b}, \mathsf{aux}_{1-b})$ and the view of $\mathcal{A}$.

The *ideal-world* execution with a stateful simulator $\mathcal{S}$ and stateful adversary $\mathcal{A}$ corrupting P_b is defined as follows:

1) $\mathcal{A}$ outputs p.
2) $\mathcal{S}(b)$ outputs k_b, which is given to $\mathcal{A}$.
3) $\mathcal{A}$ outputs x.
4) Allow $\mathcal{S}$ to interact with[2] $\mathcal{A}$, after which $\mathcal{S}$ outputs y_b, aux_b. Send x, y_b, aux_b to an ideal functionality that computes $y_{1-b} := g_p(x) - y_b$ as well as $\mathsf{aux}_{1-b} \leftarrow \mathcal{S}_{\mathsf{AUX}}(b, p, \mathsf{aux}_b)$.

We let $\text{IDEAL}_{\mathcal{S},\mathcal{A}}(1^\lambda)$ be the random variable corresponding to $(y_{1-b}, \mathsf{aux}_{1-b})$ and the view of $\mathcal{A}$.

Definition 4. *An IFSS scheme is secure if there is an efficient simulator $\mathcal{S}$ such that for any efficient adversary $\mathcal{A}$ the distributions $\text{REAL}_{\mathcal{A}}(1^\lambda)$ and $\text{IDEAL}_{\mathcal{S},\mathcal{A}}(1^\lambda)$ are computationally indistinguishable.*

Note that FSS schemes correspond to the special case of IFSS schemes where AUX is the empty function and Π involves only local computation (and no interaction). Also, a similar relaxation to allow multiple rounds was done in some semi-honest prior works [38], [62]. However, we formalize this relaxation while supporting active security as well.

3. FSS-Based 2PC

In this section we provide an overview of our approach for constructing an actively secure two-party FSS-based protocol. We begin by reviewing prior work, and then discuss the differences of our approach. In what follows, we view the circuit for $\mathcal{M}$ being computed by the parties as a sequence of *gates*, with *wires* connecting one gate to the next. We assume the circuit is fixed and known to all parties, including the dealer. A gate can have one or more input wires, and can represent a complex function and not just traditional AND/OR gates.

3.1. Prior Work

The semi-honest FSS-based protocol [6], [9] from prior work assumes all wires hold values in $\mathbb{Z}_N$. It maintains the invariant that, for each wire, the parties hold a masked wire value for a random mask chosen by the dealer. That is, for a given wire with value $x \in \mathbb{Z}_N$ when the circuit is evaluated on the parties' inputs, the parties hold the common value $\hat{x} = x + \mathsf{r} \in \mathbb{Z}_N$ for a random mask $\mathsf{r} \in \mathbb{Z}_N$ chosen by the dealer. It is easy to establish this invariant at the input wires: the dealer sends mask r to the party holding input x, and that party sends $\hat{x} = x + \mathsf{r} \bmod N$ to the other. To maintain the invariant across a gate computing a function g, the parties use an FSS scheme for the function family

$$\{g_{\mathsf{r}_{\mathsf{in}}, \mathsf{r}_{\mathsf{out}}}(\hat{x}) = g(\hat{x} - \mathsf{r}_{\mathsf{in}}) + \mathsf{r}_{\mathsf{out}}\}_{\mathsf{r}_{\mathsf{in}}, \mathsf{r}_{\mathsf{out}}},$$

2. Note that we do not allow $\mathcal{S}$ to rewind $\mathcal{A}$.

where appropriate FSS keys for $g_{\mathsf{r_{in}},\mathsf{r_{out}}}$ are given to them by the dealer (who knows $\mathsf{r_{in}}, \mathsf{r_{out}}$). The parties learn the final output by exchanging their shares of the output wires.

Boyle et al. [6, Appendix B] suggest a way to modify the above to obtain an actively secure protocol. Essentially, the invariant now is that parties hold masked, authenticated versions of the wire values; thus, masked values now lie in the larger ring $\mathbb{Z}_{\widetilde{N}}$ rather than $\mathbb{Z}_N$. Roughly speaking, the invariant is maintained at a gate by evaluating an underlying FSS scheme twice: once to compute the masked value and once to compute the masked tag for the value. The final output is obtained using a generic actively secure 2PC protocol for a relatively simple function that performs a batch correctness check of all authenticated values used throughout the protocol.

3.2. Our Approach

For our actively secure protocol, we use the original invariant from the semi-honest setting, so that for a given wire value both parties hold the masked wire value. To prevent a malicious party from modifying these values, we authenticate the masked values (rather than masking the authenticated values as in [6]). The fact that our masked values are only n-bits long means that the underlying (I)FSS schemes we use can operate on inputs that are only n-bits long, allowing for improved computational efficiency and reduced preprocessing material.

Before giving a more-detailed description of our protocol, we remark that we improve efficiency as compared to the actively secure protocol of Boyle et al. in two other ways. First, we allow the underlying circuit to have wires holding boolean values in addition to wires holding arithmetic values in $\mathbb{Z}_N$. Moreover, motivated by recent work on FSS-based protocols in the semi-honest setting [34], [38], we also use *interactive* FSS schemes to maintain the desired invariant across gates rather than relying only on (non-interactive) FSS schemes. This allows us to handle gates for computing various functions commonly used for ML inference more efficiently than in prior work (as we describe in Section 5).

We now describe our overall protocol in detail. Although our description mentions preprocessing material sent by the dealer throughout the protocol, this is done only for simplicity of exposition and all required material can be sent by the dealer during an input-independent offline phase.

Setup. The dealer chooses uniform $\Delta_0^{\mathsf{A}}, \Delta_1^{\mathsf{A}} \in \mathbb{Z}_{2^s}$ and $\Delta_0^{\mathsf{B}}, \Delta_1^{\mathsf{B}} \in \{0,1\}_\kappa$ and sends $\Delta_b^{\mathsf{A}}, \Delta_b^{\mathsf{B}}$ to P_b. The dealer sets $\Delta^{\mathsf{A}} := \Delta_0^{\mathsf{A}} + \Delta_1^{\mathsf{A}} \bmod \widetilde{N}$ and $\Delta^{\mathsf{B}} := \Delta_0^{\mathsf{B}} \oplus \Delta_1^{\mathsf{B}} \bmod \widetilde{N}$.

For each boolean wire in the circuit the dealer samples a uniform mask $\mathsf{r} \in \{0,1\}$, and for each arithmetic wire in the circuit it samples a uniform mask $\tilde{\mathsf{r}} \in \mathbb{Z}_{\widetilde{N}}$. For an arithmetic mask, we write r for the value $\mathsf{r} = \tilde{\mathsf{r}} \bmod N$.

Input. For a boolean input from P_b the dealer sends P_b the corresponding wire mask $\mathsf{r} \in \{0,1\}$, and for an arithmetic input the dealer sends the corresponding wire mask $\mathsf{r} = \tilde{\mathsf{r}} \bmod N$. Once its input x on a wire is known, P_b sends $\hat{x} = x \oplus \mathsf{r}$ or $\hat{x} = x + \mathsf{r} \bmod N$, respectively, to P_{1-b},

Computation. We describe how parties securely evaluate a single-input gate $g : \mathbb{Z}_N \to \mathbb{Z}_N$ in the underlying circuit; the process is analogous if g involves multiple input wires. (The case of boolean inputs is entirely analogous; boolean outputs are simpler to handle than arithmetic outputs since there is no need to embed values in a larger ring for authentication.) Let $\mathsf{r_{in}} \in \mathbb{Z}_N$ (where $\mathsf{r_{in}} = \tilde{\mathsf{r}}_{\mathsf{in}} \bmod N$) be the mask used by the dealer on the input wire to the gate, and let $\tilde{\mathsf{r}}_{\mathsf{out}} \in \mathbb{Z}_{\widetilde{N}}$ be the mask for the output wire. The invariant ensures that both parties know $\hat{x} = x + \mathsf{r_{in}} \in \mathbb{Z}_N$, where $x \in \mathbb{Z}_N$ is the underlying wire value when the circuit is evaluated on the parties' inputs. In addition, the dealer provides the parties with (I)FSS keys for computing the keyed function

$$g_{\mathsf{r_{in}},\tilde{\mathsf{r}}_{\mathsf{out}},\Delta^{\mathsf{A}}} : \mathbb{Z}_N \to \mathbb{Z}_{\widetilde{N}} \times \mathbb{Z}_{\widetilde{N}},$$

where $g_{\mathsf{r_{in}},\tilde{\mathsf{r}}_{\mathsf{out}},\Delta^{\mathsf{A}}}(\hat{x}) = \tilde{y} \| \Delta^{\mathsf{A}} \cdot \tilde{y}$ with

$$\tilde{y} = g(\hat{x} - \mathsf{r_{in}}) + \tilde{\mathsf{r}}_{\mathsf{out}} \bmod \widetilde{N}.$$

(In the case of IFSS there is also an auxiliary function AUX that we discuss below.) The parties evaluate the (I)FSS scheme with their respective keys and input $\hat{x}$ to obtain shares of $\tilde{y} \| \Delta^{\mathsf{A}} \cdot \tilde{y}$; note this is equivalent to having the parties hold $[\![\hat{y}]\!]$ for $\hat{y} = \tilde{y} \bmod N$. Say P_0 holds $(\tilde{y}_0, \mathsf{t}_0)$ and P_1 holds $(\tilde{y}_1, \mathsf{t}_1)$. The parties exchange $\tilde{y}_0, \tilde{y}_1 \in \mathbb{Z}_{\widetilde{N}}$ (deferring the authenticity check to the end of the protocol) and then set $\hat{y} := \tilde{y}_0 + \tilde{y}_1 \bmod N$. Note that $\hat{y} = g(x) + \mathsf{r_{out}}$ where $\mathsf{r_{out}} = \tilde{\mathsf{r}}_{\mathsf{out}} \bmod N$, so this establishes the invariant on the output wire of g.

Whenever IFSS is used in the above process, the associated function AUX will output the values exchanged during the parties' interaction along with shares of tags on those values. The authenticity check on those tags is defered to the end of the protocol (see next).

Output. At the end of the above process, the parties hold masked output values. Before reconstructing the output, the parties first check correctness of all the masked wire values, as well as any auxiliary values output from executions of the IFSS schemes, using batch checking (cf. Section 2.2). (The batch checks on the intermediate values cannot be combined with the batch checks on the output values since revealing the output when an intermediate share was incorrect may leak information.) Two batch checks are run, one for arithmetic values and one for boolean values. If either check fails, the parties abort. Otherwise, the parties proceed as discussed next.

To reconstruct a masked output value $\hat{x} = x + \mathsf{r}$, the dealer provides the parties with $[\![-\mathsf{r}]\!]$, an authenticated sharing of $-\mathsf{r}$. The parties homomorphically compute $[\![x]\!] = [\![-\mathsf{r}]\!] + \hat{x}$, and then exchange their shares for all output wire values, verifying correctness using another batch check. An analogous procedure is used for boolean output values.

We provide a security proof for the overall protocol in Appendix A.

4. Basic FSS Schemes

We begin by briefly discussing FSS schemes that are either explicit in, or can be easily constructed based on, prior work. We let $N = 2^n$, $\tilde{N} = 2^{n+s}$, $L = 2^\ell$, and $\tilde{L} = 2^{\ell+s}$.

4.1. Distributed Point Function (DPF)

The family of point functions is

$$\mathcal{G}^\bullet = \{g^\bullet_{\alpha,\beta} : \mathbb{Z}_N \to \mathbb{Z}_L\}_{\alpha\in\mathbb{Z}_N,\beta\in\mathbb{Z}_L},$$

where $g^\bullet_{\alpha,\beta}(x) = \beta \cdot \mathbb{1}\{x = \alpha\}$. That is, the output is equal to a specified payload β on a specified point α (and is 0 everywhere else). A distributed point function (DPF) is an FSS scheme for $\mathcal{G}^\bullet$. Boyle et al. [8] show a construction $\mathsf{DPF}_{n,\ell} = (\mathsf{Gen}^\bullet_{n,\ell}, \mathsf{Eval}^\bullet_{n,\ell})$ from any pseudorandom generator $G : \{0,1\}^\lambda \to \{0,1\}^{2\lambda+2}$ such that

- Each key output by $\mathsf{Gen}^\bullet_{n,\ell}$ is $n(\lambda+2)+\lambda+\ell$ bits.
- If $\ell \le \lambda$, then $\mathsf{Gen}^\bullet_{n,\ell}$ (resp., $\mathsf{Eval}^\bullet_{n,\ell}$) makes $2n$ (resp., n) calls to G.
- If $\ell > \lambda$, then $\mathsf{Gen}^\bullet_{n,\ell}$ (resp., $\mathsf{Eval}^\bullet_{n,\ell}$) makes $2n+\ell'$ (resp., $n+\ell'$) calls to G, where $\ell' = \lceil \ell/(2\lambda+2) \rceil$.

For $\lambda = 126$, it is possible to implement G using four calls to fixed-key AES [67]. Moreover, when G is instantiated in this way, the scheme can be optimized so the number of AES calls in Eval is further cut in half. Overall, at the 126-bit security level, it is thus possible to implement $\mathsf{Gen}^\bullet_{n,\ell}$ (resp., $\mathsf{Eval}^\bullet_{n,\ell}$) using only $4n$ (resp., n) calls to fixed-key AES when $\ell \le \lambda$, and $4n+2\lceil \ell/128 \rceil$ (resp., $n+\lceil \ell/128 \rceil$) calls to fixed-key AES when $\ell > \lambda$.

It is often useful to evaluate a DPF at all points in its domain; we refer to the resulting algorithm as $\mathsf{EvalAll}^\bullet_{n,\ell}$. In this case additional optimizations can be applied so that when $\ell \le \lambda$ then $\mathsf{EvalAll}^\bullet_{n,\ell}$ makes $2(2^n-1)$ AES calls, and if $\ell > \lambda$ then $\mathsf{EvalAll}^\bullet_{n,\ell}$ makes $2(2^n-1)+2^n\ell'$ AES calls, where $\ell' = \lceil \ell/128 \rceil$.

It is easy to authenticate the output by setting the payload equal to $\tilde{\beta}\|\Delta^{\mathsf{A}} \cdot \tilde{\beta}$, where $\tilde{\beta} \in \mathbb{Z}_{\tilde{L}}$ is uniform subject to $\tilde{\beta} = \beta \bmod L$. This corresponds to an FSS scheme for the function family

$$\mathcal{G}^\bullet_{\mathsf{auth}} = \{g^\bullet_{\alpha,\tilde{\beta},\Delta^{\mathsf{A}}} : \mathbb{Z}_N \to \mathbb{Z}_{\tilde{L}} \times \mathbb{Z}_{\tilde{L}}\},$$

where $g^\bullet_{\alpha,\tilde{\beta},\Delta^{\mathsf{A}}}(\hat{x}) = \tilde{\beta}\|\Delta^{\mathsf{A}} \cdot \tilde{\beta}$ if $\hat{x} = \alpha$ and $0||0$ otherwise. (Here, $\alpha \in \mathbb{Z}_N$, $\tilde{\beta} \in \mathbb{Z}_{\tilde{L}}$, and $\Delta^{\mathsf{A}} \in \mathbb{Z}_{\tilde{L}}$.) We write $\mathsf{SPDZ\text{-}DPF}_{n,\ell} = (\mathsf{SPDZ\text{-}Gen}^\bullet_{n,\ell}, \mathsf{SPDZ\text{-}Eval}^\bullet_{n,\ell})$ for the resulting FSS scheme. Similarly, we write $\mathsf{SPDZ\text{-}EvalAll}^\bullet_{n,\ell}$ for the corresponding algorithm that evaluates its given key on all inputs in $\mathbb{Z}_N$.

4.2. Distributed Comparison Function (DCF)

Consider the family of comparison functions

$$\mathcal{G}^<_{n,\ell} = \{g^<_{\alpha,\beta} : \mathbb{Z}_N \to \mathbb{Z}_L\}_{\alpha\in\mathbb{Z}_N,\beta\in\mathbb{Z}_L},$$

where $g^<_{\alpha,\beta}(x) = \beta \cdot \mathbb{1}\{x < \alpha\}$. A distributed comparison function is an FSS scheme for $\mathcal{G}^<_{n,\ell}$. Boyle et al. [6] show a construction $\mathsf{DCF}_{n,\ell} = (\mathsf{Gen}^<_{n,\ell}, \mathsf{Eval}^<_{n,\ell})$ from any pseudorandom generator $G : \{0,1\}^\lambda \to \{0,1\}^{4\lambda+2}$ such that:

- Each key output by $\mathsf{Gen}^<_{n,\ell}$ is $n(\lambda+\ell+2)+\lambda+\ell$ bits.
- If $\ell \le \lambda$, then $\mathsf{Gen}^<_{n,\ell}$ (resp., $\mathsf{Eval}^<_{n,\ell}$) makes $2n$ (resp., n) calls to G.
- If $\ell > \lambda$, then $\mathsf{Gen}^<_{n,\ell}$ (resp., $\mathsf{Eval}^<_{n,\ell}$) makes $2n(1+\lceil \ell/(4\lambda+2) \rceil)$ (resp., $n(1+\lceil \ell/(4\lambda+2) \rceil)$) calls to G.

For $\lambda = 126$, it is possible to implement G using four calls to fixed-key AES [67]. Moreover, when G is instantiated in that way, the scheme can be optimized so the number of AES calls in $\mathsf{Eval}^<$ is cut in half. Overall, at the 126-bit security level, it is possible to implement $\mathsf{Gen}^<_{n,\ell}$ (resp., $\mathsf{Eval}^<_{n,\ell}$) using only $8n$ (resp., $2n$) calls to fixed-key AES when $\ell \le \lambda$, and $(8+2\lceil \ell/128 \rceil)n$ (resp., $(2+\lceil \ell/128 \rceil)n$) calls to fixed-key AES when $\ell > \lambda$.

We can authenticate the output of a comparison function (with 1-bit output) by setting $\beta = 1\|\Delta^{\mathsf{B}} \in \{0,1\}^{\kappa+1}$. We write $\mathsf{FKOS\text{-}DCF}_n = (\mathsf{FKOS\text{-}Gen}^<_n, \mathsf{FKOS\text{-}Eval}^<_n)$ for the corresponding FSS scheme. When $\mathsf{SPD}\mathbb{Z}_{2^k}$-style arithmetic shares are needed (i.e. $[\![1]\!]$ when $x < \alpha$, and $[\![0]\!]$ otherwise), we use a transformation as in Section 4.1 and refer to the resulting scheme as $\mathsf{SPDZ\text{-}DCF}_{n,\ell} = (\mathsf{SPDZ\text{-}Gen}^<_{n,\ell}, \mathsf{SPDZ\text{-}Eval}^<_{n,\ell})$.

4.3. Linear Layers

In neural networks, linear layers involve matrix multiplications, convolutions, and dot products. FSS schemes for authenticated, masked versions of each of these can be built from a basic scheme for authenticated multiplication of masked inputs. That is, consider the function family

$$\{\mathsf{Mul}_{\mathsf{r}^{(1)}_{\mathsf{in}},\mathsf{r}^{(2)}_{\mathsf{in}},\tilde{\mathsf{r}}_{\mathsf{out}},\Delta^{\mathsf{A}}} : \mathbb{Z}_N \times \mathbb{Z}_N \to \mathbb{Z}_{\widetilde{N}} \times \mathbb{Z}_{\widetilde{N}}\},$$

where $\mathsf{Mul}_{\mathsf{r}^{(1)}_{\mathsf{in}},\mathsf{r}^{(2)}_{\mathsf{in}},\tilde{\mathsf{r}}_{\mathsf{out}},\Delta^{\mathsf{A}}}(\hat{x},\hat{y}) = \tilde{z}\|\Delta^{\mathsf{A}} \cdot \tilde{z}$ for

$$\tilde{z} = (\hat{x} - \mathsf{r}^{(1)}_{\mathsf{in}}) \cdot (\hat{y} - \mathsf{r}^{(2)}_{\mathsf{in}}) + \tilde{\mathsf{r}}_{\mathsf{out}} \bmod \widetilde{N}.$$

An FSS scheme for this function family can be constructed by suitably modifying the actively secure multiplication protocol from $\mathsf{SPD}\mathbb{Z}_{2^k}$ [19] based on Beaver triples [3]. This requires preprocessing material of $6(n+s)$ bits.

Consider next the analogous problem of multiplication of the (masked) matrices $\hat{X} \in \mathbb{Z}_N^{d_0\times d_1}$ and $\hat{Y} \in \mathbb{Z}_N^{d_1\times d_2}$. This can immediately be reduced to $d_0d_1d_2$ multiplications in $\mathbb{Z}_N$, giving an FSS-based protocol with $6d_0d_1d_2(n+s)$ bits of preprocessing. Adapting ideas from Chen et al. [18] (which itself builds on SecureML [53]), and using Beaver *matrix* triples instead of Beaver *multuplication* triples, we can obtain an FSS scheme that uses $2(d_0d_1 + d_1d_2 + d_0d_2)(n+s)$ bits of preprocessing. See Appendix B.1.

4.4. Look-up Tables

Consider a table look-up on masked inputs. That is, for a public table $\mathbf{T} \in \mathbb{Z}_L^N$, consider the function family

$$\mathsf{LUT}_{n,\ell,\mathbf{T}} = \{\mathsf{LUT}_{\mathsf{r}_{\mathsf{in}},\tilde{\mathsf{r}}_{\mathsf{out}},\Delta^{\mathsf{A}}} : \mathbb{Z}_N \to \mathbb{Z}_{\tilde{L}} \times \mathbb{Z}_{\tilde{L}}\}_{\mathsf{r}_{\mathsf{in}},\tilde{\mathsf{r}}_{\mathsf{out}},\Delta^{\mathsf{A}}},$$

with $\mathsf{LUT}_{\mathsf{r}_{\mathsf{in}},\tilde{\mathsf{r}}_{\mathsf{out}},\Delta^{\mathsf{A}}}(\hat{x}) = \tilde{y} \| \Delta^{\mathsf{A}} \cdot \tilde{y}$ for

$$\tilde{y} = \mathbf{T}[\hat{x} - \mathsf{r}_{\mathsf{in}}] + \tilde{\mathsf{r}}_{\mathsf{out}}.$$

We provide the details of an FSS scheme for this family in Appendix B.2. (Pika [65] shows an FSS scheme for look-up tables in a stronger threat model where the dealer can be malicious. Our protocol is equivalent to theirs if the dealer-specific checks are removed.) The scheme requires preprocessing material of size $n(\lambda+2)+\lambda+4(\ell+s)$ bits, and evaluation uses $(2+\lceil 2(\ell+s)/128 \rceil)\cdot 2^n - 2$ calls to AES.

5. Schemes for Non-Linear Functions

In this section, we show (I)FSS schemes for (the masked and authenticated versions of) the non-linear functions ReLU, ARS/truncation, ReluARS (a fused ReLU and truncation operation), and splines (used to approximate transcendental functions like sigmoid, tanh, etc.). We also show an approach for computing reciprocals (used for computing softmax in transformers) based on other (I)FSS schemes.

In all our IFSS schemes, the auxiliary information (aux_0, aux_1) output by the parties consists of uniformly distributed elements opened by the parties during their interaction (revealed to both parties) along with their corresponding authentication tags (shared between the parties and not used until the batch check at the end of the overall protocol). For clarity we thus leave AUX and this auxiliary information implicit when describing our schemes.

In all cases, when we report communication complexity of our schemes we include the reconstruction of the final output (without the associated tag).

5.1. ReLU

The function $\mathsf{ReLU}(x) = \max(x, 0)$ is an important non-linear function used in neural networks. We show here an FSS scheme for computing this function with masked inputs and authenticated, masked outputs. To do so, we rely on an insight from prior work giving passively secure FSS schemes for ReLU [34], [38]; namely, we decompose computation of ReLU into the computation of two simpler functions, and then give FSS schemes for those simpler functions. Specifically, observe that if $x \in \mathbb{Z}_N$ is expressed in two's complement representation then $\mathsf{ReLU}(x) = (1 \oplus \mathsf{MSB}(x)) \cdot x$, where $\mathsf{MSB}(x)$ is the most significant bit of x. That is, we can first compute

$$d = \mathsf{GEZ}(x) \stackrel{\text{def}}{=} 1 \oplus \mathsf{MSB}(x) = \mathbb{1}\{x < 2^{n-1}\}, \qquad (1)$$

and then use d to select between x and 0. To map this to our setting, we show FSS schemes for (masked and authenticated versions of) the comparison and selection functions.

Comparison. To implement the required FSS scheme for comparisons, we use the FKOS-DCF construction from Section 4.2 to incorporate input and output masks $\mathsf{r}_{\mathsf{in}} \in \mathbb{Z}_N$ and $\mathsf{r}_{\mathsf{tmp}} \in \{0,1\}$. Following [34], we let $\mathsf{r} = -\mathsf{r}_{\mathsf{in}} \bmod N$ and write

$$\begin{aligned}
&\mathsf{GEZ}(\hat{x} - \mathsf{r}_{\mathsf{in}}) \oplus \mathsf{r}_{\mathsf{tmp}} \\
&= 1 \oplus \mathsf{r}_{\mathsf{tmp}} \oplus \mathsf{MSB}(\hat{x} + \mathsf{r}) \\
&= 1 \oplus \mathsf{r}_{\mathsf{tmp}} \oplus \mathsf{MSB}(\hat{x}) \oplus \mathsf{MSB}(\mathsf{r}) \\
&\quad \oplus \mathbb{1}\{(\hat{x} \bmod 2^{n-1}) + (\mathsf{r} \bmod 2^{n-1}) \geq 2^{n-1}\} \\
&= \mathsf{MSB}(\hat{x}) \oplus 1 \oplus \mathsf{r}_{\mathsf{tmp}} \oplus \mathsf{MSB}(\mathsf{r}) \\
&\quad \oplus \mathbb{1}\{2^{n-1} - (\hat{x} \bmod 2^{n-1}) - 1 < (\mathsf{r} \bmod 2^{n-1})\}.
\end{aligned}$$

The first term can be computed locally; shares of the last term can be computed using a DCF; and the XOR of the middle terms can be computed in advance by the dealer and shared with the parties. See Figure 1, where we also incorporate authentication of the output.

$\mathsf{Gen}_n^{\mathsf{GEZ}}(\mathsf{r}_{\mathsf{in}}, \mathsf{r}_{\mathsf{tmp}}, \Delta^{\mathsf{B}})$: ▷ $\mathsf{r}_{\mathsf{in}} \in \mathbb{Z}_N, \mathsf{r}_{\mathsf{tmp}} \in \{0,1\}$

1: $\alpha := -\mathsf{r}_{\mathsf{in}} \bmod 2^{n-1}$
2: $(k_0^<, k_1^<) \leftarrow \mathsf{FKOS\text{-}Gen}_{n-1}^<(\alpha, \Delta^{\mathsf{B}})$
3: $r := 1 \oplus \mathsf{r}_{\mathsf{tmp}} \oplus \mathsf{MSB}(-\mathsf{r}_{\mathsf{in}})$
4: $\langle r \rangle \leftarrow \mathsf{auth\text{-}bshare}(r)$
5: for $b \in \{0,1\}$, set $k_b := (\langle r \rangle_b, k_b^<)$

$\mathsf{Eval}_n^{\mathsf{GEZ}}(k_b, \hat{x})$: ▷ $\hat{x} \in \mathbb{Z}_N$

1: Parse k_b as $(\langle r \rangle_b, k_b^<)$
2: $y := 2^{n-1} - (\hat{x} \bmod 2^{n-1}) - 1$
3: $\langle t \rangle_b \leftarrow \mathsf{FKOS\text{-}Eval}_{n-1}^<(k_b^<, y)$
4: **return** $\langle \hat{d} \rangle_b = \mathsf{MSB}(\hat{x}) \oplus \langle r \rangle_b \oplus \langle t \rangle_b$

Figure 1: FSS scheme for comparison.

Selection. We now describe an FSS scheme for selection with masked inputs/outputs. Note that

$$\begin{aligned}
&(\hat{d} \oplus \mathsf{r}_{\mathsf{tmp}}) \cdot (\hat{x} - \mathsf{r}_{\mathsf{in}}) + \tilde{\mathsf{r}}_{\mathsf{out}} \\
&\quad = \begin{cases} \mathsf{r}_{\mathsf{tmp}} \cdot \hat{x} - \mathsf{r}_{\mathsf{tmp}} \cdot \mathsf{r}_{\mathsf{in}} + \tilde{\mathsf{r}}_{\mathsf{out}} & \hat{d} = 0 \\ \hat{x} - \mathsf{r}_{\mathsf{tmp}} \cdot \hat{x} - \mathsf{r}_{\mathsf{in}} + \mathsf{r}_{\mathsf{tmp}} \cdot \mathsf{r}_{\mathsf{in}} + \tilde{\mathsf{r}}_{\mathsf{out}} & \hat{d} = 1, \end{cases}
\end{aligned}$$

where $\hat{x}, \mathsf{r}_{\mathsf{in}} \in \mathbb{Z}_N$, $\tilde{\mathsf{r}}_{\mathsf{out}} \in \mathbb{Z}_{\widetilde{N}}$, and $\mathsf{r}_{\mathsf{tmp}} \in \{0,1\}$. For known $\hat{d}$ and $\hat{x}$, the above are each a linear expression in values $\mathsf{r}_{\mathsf{tmp}}, \mathsf{r}_{\mathsf{in}}, \mathsf{r}_{\mathsf{tmp}} \cdot \mathsf{r}_{\mathsf{in}}, \tilde{\mathsf{r}}_{\mathsf{out}}$ known to the dealer. Thus, the dealer can simply provide the parties with $\mathsf{SPD}\mathbb{Z}_{2^k}$-shares of these values, and the parties can then use those shares to compute shares of the masked output. See Figure 2.

Our ReLU **protocol.** Putting everything together we obtain a 2-round FSS-based protocol for ReLU, with preprocessing material $\mathsf{keysize}(\mathsf{FKOS\text{-}DCF}_{n-1}) + 8(n+s) + (\kappa+1)$ bits long, and online communication of $n+s+1$ bits. For $n = s = 64$ and $\kappa = 40$, this is $11,879$ bits of preprocessing material per party and online communication of 129 bits. The online phase requires 126 AES calls.

Comparison to prior work. It is possible to construct an actively secure FSS-based protocol for ReLU by applying the compiler of Boyle et al. [6] to the passively secure ReLU

$\mathsf{Gen}_n^{\mathsf{select}}((\mathsf{r}_{\mathsf{tmp}}, \mathsf{r}_{\mathsf{in}}), \tilde{\mathsf{r}}_{\mathsf{out}}, \Delta^{\mathsf{A}})$: ▷ $\mathsf{r}_{\mathsf{tmp}} \in \{0,1\}$

1: $u = \mathsf{B2A}_n(\mathsf{r}_{\mathsf{tmp}})$ ▷ $\mathsf{r}_{\mathsf{in}} \in \mathbb{Z}_N, \tilde{\mathsf{r}}_{\mathsf{out}} \in \mathbb{Z}_{\tilde{N}}$
2: $v := \mathsf{r}_{\mathsf{in}},\ w := \mathsf{r}_{\mathsf{tmp}} \cdot \mathsf{r}_{\mathsf{in}},\ z := \tilde{\mathsf{r}}_{\mathsf{out}}$
3: $[\![u]\!], [\![v]\!], [\![w]\!], [\![z]\!] \leftarrow \mathsf{auth\text{-}ashare}(u, v, w, z)$
4: for $b \in \{0,1\}$, set $k_b := ([\![u]\!]_b, [\![v]\!]_b, [\![w]\!]_b, [\![z]\!]_b)$

$\mathsf{Eval}_n^{\mathsf{select}}(k_b, (\hat{d}, \hat{x}))$: ▷ $\hat{d} \in \{0,1\}, \hat{x} \in \mathbb{Z}_N$

1: Parse k_b as $([\![u]\!]_b, [\![v]\!]_b, [\![w]\!]_b, [\![z]\!]_b)$
2: **if** $\hat{d} = 0$ **then**
3: **return** $[\![\hat{y}]\!]_b := \hat{x} \cdot [\![u]\!]_b - [\![w]\!]_b + [\![z]\!]_b$
4: **else**
5: **return** $[\![\hat{y}]\!]_b := \hat{x} - \hat{x} \cdot [\![u]\!]_b - [\![v]\!]_b + [\![w]\!]_b + [\![z]\!]_b$
6: **end if**

Figure 2: FSS scheme for selection.

protocol in that same work. This gives a 1-round protocol that communicates $n + s$ bits in the online phase and uses $2 \cdot (2 + \lceil 4(n+s)/128 \rceil)(n+s)$ calls to AES. For $n = s = 64$, this results in $\approx 12\times$ more AES calls for similar online communication. It also requires preprocessing material of at least $83,838$ bits which is $\approx 7\times$ larger than our protocol.

Escudero et al. [29] show an actively secure ReLU protocol with round complexity $\Theta(\log n)$. For $n = s = 64$, their protocol requires $18,459$ bits of preprocessing material and 621 bits of online communication, which are $1.54\times$ and $4.8\times$ higher, respectively, than our protocol.

MaxPool. Neural-network inference often requires computing the maximum element in a pool of k elements. This can be reduced to $k-1$ iterated computations of the maximum of two elements, which can in turn be computed as $\mathsf{Max}(x, y) = \mathsf{ReLU}(x - y) + y$.

5.2. Truncation/Right Shifts

Multiplication of two signed (resp., unsigned), n-bit fixed-point numbers with precision f involves signed (resp., unsigned) integer multiplication of the two operands, followed by arithmetic (resp., logical) right shift of the resulting number by f positions. Hence, to support fixed-point multiplication we devise IFSS schemes for arithmetic right shift (ARS) and logical right shift (LRS). (Recall that arithmetic right shift sets the most-significant bits of the result equal to the most-significant bit of the input, whereas logical right shift sets the most-significant bits of the result to 0.) Note a protocol for ARS can be constructed from a protocol for LRS using $\mathsf{ARS}_{n,f}(x) = \mathsf{LRS}_{n,f}(x + 2^{n-1}) - 2^{n-f-1}$, where all arithmetic is done in $\mathbb{Z}_N$.

Following [34], we have

$$\begin{aligned}
&\mathsf{LRS}_{n,f}(\hat{x} - \mathsf{r}_{\mathsf{in}}) + \tilde{\mathsf{r}}_{\mathsf{out}} \\
&= \mathsf{LRS}_{n,f}(\hat{x}) - \mathsf{LRS}_{n,f}(\mathsf{r}_{\mathsf{in}}) + \tilde{\mathsf{r}}_{\mathsf{out}} \\
&\quad + 2^{n-f} \cdot \mathbb{1}\{\hat{x} < \mathsf{r}_{\mathsf{in}}\} \\
&\quad - \mathbb{1}\{(\hat{x} \bmod 2^f) < (\mathsf{r}_{\mathsf{in}} \bmod 2^f)\}.
\end{aligned}$$

The first term on the right-hand side can be computed locally by the parties. The second and third terms can be computed by the dealer. The remaining terms can be handled using DCFs. While it would be possible to use DCFs that directly output $0, 1 \in \mathbb{Z}_N$ (along with a corresponding authentication tag), prior work [38] observed that it is more efficient to use DCFs with boolean output and then run a 1-round protocol to convert those boolean output values to arithmetic values. Specifically, the parties reconstruct the masked boolean outputs from the two DCF evaluations, and then use a table look-up (with a table provided as secret shares by the dealer) to convert those to arithmetic values. As an optimization, the second and the third terms are also incorporated into the table entries. We provide the complete protocol in Figure 3, and a proof of security in Appendix C.1.

$\mathsf{Gen}_{n,f}^{\mathsf{LRS}}(\mathsf{r}_{\mathsf{in}}, \tilde{\mathsf{r}}_{\mathsf{out}}, \Delta^{\mathsf{B}}, \Delta^{\mathsf{A}})$: ▷ $\mathsf{r}_{\mathsf{in}} \in \mathbb{Z}_N, \tilde{\mathsf{r}}_{\mathsf{out}} \in \mathbb{Z}_{\tilde{N}}$

1: $(k_{n,0}^<, k_{n,1}^<) \leftarrow \mathsf{FKOS\text{-}Gen}_n^<(\mathsf{r}_{\mathsf{in}}, \Delta^{\mathsf{B}})$
2: $(k_{f,0}^<, k_{f,1}^<) \leftarrow \mathsf{FKOS\text{-}Gen}_f^<(\mathsf{r}_{\mathsf{in}} \bmod 2^f, \Delta^{\mathsf{B}})$
3: $r^{(w)}, r^{(t)} \leftarrow \{0,1\}$
4: $\langle r^{(w)} \rangle \leftarrow \mathsf{auth\text{-}bshare}(r^{(w)})$
5: $\langle r^{(t)} \rangle \leftarrow \mathsf{auth\text{-}bshare}(r^{(t)})$
6: $\delta := -\mathsf{LRS}_{n,f}(\mathsf{r}_{\mathsf{in}}) + \tilde{\mathsf{r}}_{\mathsf{out}}$
7: **for** $\hat{w}, \hat{t} \in \{0,1\}$
8: $\mathbf{T}[\hat{w}, \hat{t}] := 2^{n-f} \cdot (\hat{w} \oplus r^{(w)}) - (\hat{t} \oplus r^{(t)}) + \delta$
9: $[\![\mathbf{T}]\!] \leftarrow \mathsf{auth\text{-}ashare}(\mathbf{T})$
10: for $b \in \{0,1\}$, set $k_b := (k_{n,b}^<,\ k_{f,b}^<,\ \langle r^{(w)} \rangle_b,\ \langle r^{(t)} \rangle_b,\ [\![\mathbf{T}]\!]_b)$

$\mathsf{Eval}_{n,f}^{\mathsf{LRS}}(k_b, \hat{x})$: ▷ $\hat{x} \in \mathbb{Z}_N$

1: Parse k_b as $(k_{n,b}^<, k_{f,b}^<, \langle r^{(w)} \rangle_b, \langle r^{(t)} \rangle_b, [\![\mathbf{T}]\!]_b)$
2: $\langle w \rangle_b \leftarrow \mathsf{FKOS\text{-}Eval}_n^<(k_{n,b}^<, \hat{x})$
3: $\langle t \rangle_b \leftarrow \mathsf{FKOS\text{-}Eval}_f^<(k_{f,b}^<, \hat{x} \bmod 2^f)$
4: $\langle \hat{w} \rangle_b := \langle w \rangle_b \oplus \langle r^{(w)} \rangle_b$
5: $\langle \hat{t} \rangle_b := \langle t \rangle_b \oplus \langle r^{(t)} \rangle_b$
6: $\hat{w} \leftarrow \mathsf{auth\text{-}reconstruct}(\langle \hat{w} \rangle_b)$
7: $\hat{t} \leftarrow \mathsf{auth\text{-}reconstruct}(\langle \hat{t} \rangle_b)$
8: **return** $[\![\hat{y}]\!]_b = \mathsf{LRS}_{n,f}(\hat{x}) + [\![\mathbf{T}]\!]_b[\hat{w}, \hat{t}]$

Figure 3: IFSS scheme for LRS.

Our 1-round IFSS scheme uses $\mathsf{keysize}(\mathsf{FKOS\text{-}DCF}_n) + \mathsf{keysize}(\mathsf{FKOS\text{-}DCF}_f) + 2(1+\kappa) + 8(n+s)$ bits of preprocessing material per party, and communicates $n+s+2$ bits (including the final reconstruction step). For $n = s = 64$, $f = 16$, the scheme has $14,960$-bit keys, and makes 160 AES calls and communicates 130 bits in the online phase.

Comparison to prior work. Boyle et al. [6] show a semi-honest FSS scheme for LRS that is incompatible with their efficient approach for achieving active security. Nevertheless, we can lower bound the costs of their scheme by assuming they use a DCF with $(n+s)$-bit input and authenticated output. For $n = s = 64$, their scheme would use at least $49,790$ bits of preprocessing material, which is $3.3\times$ larger than in ours. In the online phase, they would call AES

512 times and communicate 128 bits. That is, our protocol requires 3.2× fewer AES calls at the cost of an additional round of interaction and 2 bits of additional communication.

Escudero et al. [29] show a protocol for truncation that requires $17,506$ bits of preprocessing and 482 bits of online communication for the same parameters. That is, they need 1.17× more preprocessing material and 3.7× more communication tha in our scheme. More problematic is that their protocol requires $\Theta(\log n)$ rounds.

Some prior works [53], [70] perform truncation by having parties locally truncate their shares. Recently, this approach was established as insecure [46]. However, another recent work [63] argues that local truncations are secure implementations of a functionality, which for an input $x = x_t||x_f$, leaks the truncated bits x_f. Furthermore, [63] argues that this leakage is acceptable. However, since in the worst case, local truncations reveal all of x, this leakage being acceptable remains unclear, and furthermore does not provide standard simulation-based security where only the final inference results should be learned.

5.3. ReluARS: Fused ReLU and ARS

Computing an ARS followed by a ReLU occurs frequently in ML inference. We provide here an optimized IFSS scheme for $\mathsf{ReluARS}_{n,f}(x) := \mathsf{ReLU}_n(\mathsf{ARS}_{n,f}(x))$ that requires 1.7× less preprocessing material and has 2× lower online communication (at the cost of only 2 additional AES calls) compared to sequential evaluation of our ARS and ReLU schemes. Note that while Orca [38] provides a scheme for ReluARS in the semi-honest setting that could in principle be lifted to the malicious setting, our approach is more efficient; in particular, it also yields a better semi-honest scheme for ReluARS than the one in that work.

We now give the details. Observe that $\mathsf{ARS}_{n,f}(x) = \mathsf{LRS}_{n,f}(x)$ for $x < 2^{n-1}$ and $\mathsf{ReLU}_n(x) = 0$ for $x \geq 2^{n-1}$. Putting these together gives $\mathsf{ReluARS}_{n,f}(x) = \mathsf{GEZ}(x) \cdot \mathsf{LRS}_{n,f}(x)$, where GEZ is defined in Eq. (1). Following [38],

$$\begin{aligned}&\mathsf{GEZ}(\hat{x} - \mathsf{r}_{\mathsf{in}})\\ &= \quad \mathbb{1}\{\hat{x} < \mathsf{r}_{\mathsf{in}}\} \oplus \mathbb{1}\{\hat{y} < \mathsf{r}_{\mathsf{in}}\} \oplus \mathsf{MSB}(\hat{y}),\end{aligned}$$

where $\hat{y} = \hat{x} + 2^{n-1} \bmod N$. This expression is suboptimal compared to the one used in Section 5.1 for two reasons: (1) it requires n-bit (rather than $(n-1)$-bit) comparisons, and (2) it needs two DCF evaluations instead of one. On the positive side, we already use a DCF for $\mathbb{1}\{\hat{x} < \mathsf{r}_{\mathsf{in}}\}$ as part of the ARS/LRS computation (cf. Section 5.2).

We compute ReluARS in two steps. First, we compute the comparisons required for LRS and GEZ. Specifically, we compute masked versions of $w = \mathbb{1}\{\hat{x} < \mathsf{r}_{\mathsf{in}}\}$ and $d = \mathbb{1}\{\hat{y} < \mathsf{r}_{\mathsf{in}}\} \oplus w \oplus \mathsf{MSB}(\hat{y})$ (note these can be computed using a single DCF key), as well as $t = \mathbb{1}\{\hat{x} \bmod 2^f < \mathsf{r}_{\mathsf{in}} \bmod 2^f\}$. We have

$$\begin{aligned}&\mathsf{ReluARS}_{n,f}(\hat{x} - \mathsf{r}_{\mathsf{in}}) + \tilde{\mathsf{r}}_{\mathsf{out}}\\ &= \mathsf{GEZ}_n(\hat{x} - \mathsf{r}_{\mathsf{in}}) \cdot \mathsf{LRS}_{n,f}(\hat{x} - \mathsf{r}_{\mathsf{in}}) + \tilde{\mathsf{r}}_{\mathsf{out}}\\ &= d \cdot (\mathsf{LRS}_{n,f}(\hat{x}) - \mathsf{LRS}_{n,f}(\mathsf{r}_{\mathsf{in}}) + 2^{n-f} \cdot w - t) + \tilde{\mathsf{r}}_{\mathsf{out}}\\ &= d \cdot \mathsf{LRS}_{n,f}(\hat{x}) + d \cdot (2^{n-f} \cdot w - \mathsf{LRS}_{n,f}(\mathsf{r}_{\mathsf{in}}) - t) + \tilde{\mathsf{r}}_{\mathsf{out}}.\end{aligned}$$

The parties compute the first term via a table look-up, using the masked value of d, $\hat{x}$, and a (shared) table provided by the dealer. They can similarly compute the second and third terms using a secret shared table provided by the dealer, indexed by masked values of w, d, and t. See Figure 4. We prove security in Appendix C.2.

Our IFSS scheme uses a total of $\mathsf{keysize}(\mathsf{FKOS\text{-}DCF}_n) + \mathsf{keysize}(\mathsf{FKOS\text{-}DCF}_f) + 3(1+\kappa) + 20(n+s)$ bits of preprocessing material per party, and requires online communication of $n+s+3$ bits per party. For $n = s = 64, f = 16$ and $\kappa = 40$, it requires $16,537$ bits of preprocessing material, 131 bits of online communication, and 288 AES calls.

Comparison to prior work. Neither Boyle et al. [6] nor Escudero et al. [29] give an optimized protocol for computing ReluARS; instead, they compute ReluARS by sequentially composing potocols for ReLU and truncation. For Boyle et al., this results in a protocol that requires at least $133,628$

$\mathsf{Gen}^{\mathsf{ReluARS}}_{n,f}(\mathsf{r}_{\mathsf{in}}, \tilde{\mathsf{r}}_{\mathsf{out}}, \Delta^{\mathsf{B}}, \Delta^{\mathsf{A}})$: ▷ $\mathsf{r}_{\mathsf{in}} \in \mathbb{Z}_N, \mathsf{r}_{\mathsf{out}} \in \mathbb{Z}_{\widetilde{N}}$

1: $(k^<_{n,0}, k^<_{n,1}) \leftarrow \mathsf{FKOS\text{-}Gen}^<_n(\mathsf{r}_{\mathsf{in}}, \Delta^{\mathsf{B}})$
2: $(k^<_{f,0}, k^<_{f,1}) \leftarrow \mathsf{FKOS\text{-}Gen}^<_f(\mathsf{r}_{\mathsf{in}} \bmod 2^f, \Delta^{\mathsf{B}})$
3: $\mathsf{r}^{(d)}, \mathsf{r}^{(w)}, \mathsf{r}^{(t)} \leftarrow \{0,1\}$
4: $\langle \mathsf{r}^{(d)} \rangle, \langle \mathsf{r}^{(w)} \rangle, \langle \mathsf{r}^{(t)} \rangle \leftarrow \mathsf{auth\text{-}bshare}(\mathsf{r}^{(d)}, \mathsf{r}^{(w)}, \mathsf{r}^{(t)})$
5: **for** $\hat{d}, \hat{w}, \hat{t} \in \{0,1\}$
6: $\quad \mathbf{T}[\hat{d}, \hat{w}, \hat{t}] := \mathsf{B2A}_n(\hat{d} \oplus \mathsf{r}^{(d)}) \cdot (-\mathsf{LRS}_{n,f}(\mathsf{r}_{\mathsf{in}})$
7: $\quad\quad + 2^{n-f} \cdot \mathsf{B2A}_n(\hat{w} \oplus \mathsf{r}^{(w)}) - \mathsf{B2A}_n(\hat{t} \oplus \mathsf{r}^{(t)})) + \tilde{\mathsf{r}}_{\mathsf{out}}$
8: **for** $\hat{d}$ in $\{0,1\}$
9: $\quad \mathbf{S}[\hat{d}] := \mathsf{B2A}_n(\hat{d} \oplus \mathsf{r}^{(d)})$
10: $[\![\mathbf{T}]\!], [\![\mathbf{S}]\!] \leftarrow \mathsf{auth\text{-}ashare}(\mathbf{T}, \mathbf{S})$
11: for $b \in \{0,1\}$, set $k_b := (k^<_{n,b}, k^<_{f,b}, \langle \mathsf{r}^{(d)} \rangle_b, \langle \mathsf{r}^{(w)} \rangle_b, \langle \mathsf{r}^{(t)} \rangle_b, [\![\mathbf{T}]\!]_b, [\![\mathbf{S}]\!]_b)$

$\mathsf{Eval}^{\mathsf{LRS}}_{n,f}(k_b, \hat{x})$: ▷ $\hat{x} \in \mathbb{Z}_N$

1: Parse k_b as $(k^<_{n,b}, k^<_{f,b}, \langle \mathsf{r}^{(d)} \rangle_b, \langle \mathsf{r}^{(w)} \rangle_b, \langle \mathsf{r}^{(t)} \rangle_b, [\![\mathbf{T}]\!]_b, [\![\mathbf{S}]\!]_b)$
2: $\hat{y} := \hat{x} + 2^{n-1} \bmod 2^n$
3: $\langle w \rangle_b \leftarrow \mathsf{FKOS\text{-}Eval}^<_n(k^<_{n,b}, \hat{x})$
4: $\langle d \rangle_b \leftarrow \mathsf{FKOS\text{-}Eval}^<_n(k^<_{n,b}, \hat{y}) \oplus \mathsf{MSB}(\hat{y}) \oplus \langle w \rangle_b$
5: $\langle t \rangle_b \leftarrow \mathsf{FKOS\text{-}Eval}^<_f(k^<_{f,b}, \hat{x} \bmod 2^f)$
6: $\langle \hat{d} \rangle_b := \langle d \rangle_b \oplus \langle \mathsf{r}^{(d)} \rangle_b$
7: $\langle \hat{w} \rangle_b := \langle w \rangle_b \oplus \langle \mathsf{r}^{(w)} \rangle_b$
8: $\langle \hat{t} \rangle_b := \langle t \rangle_b \oplus \langle \mathsf{r}^{(t)} \rangle_b$
9: $\hat{d}, \hat{w}, \hat{t} \leftarrow \mathsf{auth\text{-}reconstruct}(\langle \hat{d} \rangle_b, \langle \hat{w} \rangle_b, \langle \hat{t} \rangle_b)$
10: **return** $[\![\hat{y}]\!]_b := [\![\mathbf{S}]\!]_b[\hat{d}] \cdot \mathsf{LRS}_{n,f}(\hat{x}) + [\![\mathbf{T}]\!]_b[\hat{d}, \hat{w}, \hat{t}]$

Figure 4: IFSS scheme for ReluARS.

bits of preprocessing, 256 bits of online communication, and 2048 AES calls, which are $8\times$, $1.9\times$, and $7.1\times$ larger, respectively, than in our scheme (which has the same number of rounds). For Escudero et al., this gives a $\Theta(\log n)$-round protocol using $35,965$ bits of preprocessing material and $1,103$ bits of online communication, which are $2.2\times$ and $8.4\times$ larger, respectively, than in our scheme.

5.4. Spline

Complex mathematical functions (e.g., sigmoid, tanh, etc.) are common in many machine-learning architectures. To implement these securely, prior work [6], [35], [48], [55], [64], [71] has approximated them using splines (aka piecewise polynomials). An (m,d)-*spline* splits the input domain into m intervals and approximates the target function in each interval by a degree-d polynomial. Concretely, for degree-d polynomials $\mathbf{P} = (\mathbf{p}_0, \mathbf{p}_1 \dots \mathbf{p}_{m-1}) \in (\mathbb{Z}_N[X])^m$, a sorted list $\mathbf{H} = (h_0, \dots, h_m) \in \mathbb{Z}_{N+1}^{m+1}$ with $h_0 = 0$ and $h_m = 2^n$, and an input $x \in \mathbb{Z}_N$, define $\mathsf{spline}_{n,m,d,\mathbf{P},\mathbf{H}}(x) = \mathbf{p}_i(x)$, where $i \in [m]$ is such that $h_i \le x < h_{i+1}$.

Using secret sharing-based approaches (e.g., [29]) for splines requires the size of the preprocessing material, as well as the online computation and communication, to grow linearly with the number of intervals m. A sequence of works on semi-honest FSS-based protocols [6], [35], [64] culminating in Grotto [64] provides a protocol where both the preprocessing size and online communication are independent of m. However, it is not known how to extend Grotto to the malicious setting, and in particular the technique of Boyle et al. [6] does not apply. Hence, the only known actively secure FSS-based protocol for spline is the one obtained by applying those techniques to the semi-honest spline protocol from the same work. However, the resulting scheme requires large preprocessing material and high online computation. Below, we describe a new actively secure IFSS scheme for splines that achieves complexity similar to Grotto, and hence is much more efficient than existing actively secure schemes.

Evaluating a spline on an input x involves two steps: (1) determining the correct interval, and then (2) evaluating the corresponding polynomial on x. In our setting, where the parties begin holding masked input $\hat{x} = x + \mathsf{r}_{\mathsf{in}} \bmod N$, the parties first calculate shares of a 1-hot vector $\mathbf{u} \in \mathbb{Z}_N^m$ such that $\mathbf{u}[i] = 1$ iff $h_i \le x < h_{i+1}$. Letting $h'_i = \hat{x} - h_i \bmod N$, this is equivalent to

$$\begin{aligned}\mathbf{u}[i] &= \begin{cases} \mathbb{1}\{\mathsf{r}_{\mathsf{in}} \in (h'_{i+1}, h'_i]\} & \text{if } h'_i > h'_{i+1} \\ \mathbb{1}\{\mathsf{r}_{\mathsf{in}} \in (h'_{i+1}, N-1] \cup [0, h'_i]\} & \text{otherwise} \end{cases} \\ &= \mathbb{1}\{h'_{i+1} < \mathsf{r}_{\mathsf{in}}\} - \mathbb{1}\{h'_i < \mathsf{r}_{\mathsf{in}}\} + \mathbb{1}\{h'_i \le h'_{i+1}\}.\end{aligned}$$

The term $\mathbb{1}\{h'_i \le h'_{i+1}\}$ can be calculated locally by the parties. The terms $\mathbb{1}\{\mathsf{r}_{\mathsf{in}} > h'_{i+1}\}$ and $\mathbb{1}\{\mathsf{r}_{\mathsf{in}} > h'_i\}$ can be calculated using a single DCF key; in fact, the required comparisons for all $i \in [m]$ can be done using multiple evaluations of a DCF using the same key each time.

Given $\mathsf{SPD}\mathbb{Z}_{2^k}$-shares of $\mathbf{u}$, the parties can select the correct polynomial by locally computing the dot product $\mathbf{q} = \sum_{i=0}^{m-1} \mathbf{u}[i] \cdot \mathbf{p}_i$. We then have the parties reconstruct the masked polynomial $\hat{\mathbf{q}} = \mathbf{q} + \mathbf{r}^{(\mathbf{q})}$ for a mask $\mathbf{r}^{(\mathbf{q})}$ provided by the dealer as $\mathsf{SPD}\mathbb{Z}_{2^k}$-shares.

The parties now need to evaluate the masked polynomial on the masked input. Ignoring the output masking for simplicity, this entails computing

$$\begin{aligned}&\sum_{j=0}^{d} (\hat{\mathbf{q}}[j] - \mathbf{r}^{(\mathbf{q})}[j]) \cdot (\hat{x} - \mathsf{r}_{\mathsf{in}})^j \\ &= \sum_{j=0}^{d} \sum_{k=0}^{j} \binom{j}{k} \hat{x}^{j-k} \hat{\mathbf{q}}[j] (-\mathsf{r}_{\mathsf{in}})^k \\ &\quad - \sum_{k=0}^{d} \hat{x}^k \sum_{j=k}^{d} \binom{j}{k} (-\mathsf{r}_{\mathsf{in}})^{j-k} \mathbf{r}^{(\mathbf{q})}[j].\end{aligned}$$

For $k \in [d+1]$, define $\mathbf{S}[k] = (-\mathsf{r}_{\mathsf{in}})^k \bmod N$ and $\mathbf{T}[k] = \sum_{j=k}^{d} \binom{j}{k} (-\mathsf{r}_{\mathsf{in}})^{j-k} \mathbf{r}^{(\mathbf{q})}[j] \bmod N$, and note that those values are all known to the dealer. The above expression is then a linear expression in those values. Hence, the dealer can provide $\mathsf{SPD}\mathbb{Z}_{2^k}$-shares of $\mathbf{T}$ and $\mathbf{S}$, and the parties can then homomorphically compute (shares of) the above expression. We combine the above ideas in Figure 5.

$\mathsf{Gen}^{\mathsf{spline}}_{n,m,d,\mathbf{p},\mathbf{h}}(\mathsf{r}_{\mathsf{in}}, \tilde{\mathsf{r}}_{\mathsf{out}}, \Delta^{\mathsf{A}})$: ▷ $\mathsf{r}_{\mathsf{in}} \in \mathbb{Z}_N, \tilde{\mathsf{r}}_{\mathsf{out}} \in \mathbb{Z}_{\tilde{N}}$

1: $(k_0^<, k_1^<) \leftarrow \mathsf{SPDZ\text{-}Gen}^<_{n,n}(\mathsf{r}_{\mathsf{in}}, \Delta^{\mathsf{A}})$
2: $\mathbf{r}^{(\mathbf{q})} \leftarrow \mathbb{Z}_N^{d+1}$
3: **for** $k \in [d+1]$
4: $\quad \mathbf{S}[k] := (-\mathsf{r}_{\mathsf{in}})^k \bmod N$
5: $\quad$ **if** $k = 0$
6: $\quad\quad \mathbf{T}[k] := -\sum_{j=k}^{d} \binom{j}{k} \cdot \mathbf{r}^{(\mathbf{q})}[j] \cdot (-\mathsf{r}_{\mathsf{in}})^{j-k} + \tilde{\mathsf{r}}_{\mathsf{out}}$
7: $\quad$ **else**
8: $\quad\quad \mathbf{T}[k] := -\sum_{j=k}^{d} \binom{j}{k} \cdot \mathbf{r}^{(\mathbf{q})}[j] \cdot (-\mathsf{r}_{\mathsf{in}})^{j-k}$
9: $[\![\mathbf{r}^{(\mathbf{q})}]\!], [\![\mathbf{S}]\!], [\![\mathbf{T}]\!] \leftarrow \mathsf{auth\text{-}ashare}(\mathbf{r}_q, \mathbf{S}, \mathbf{T})$
10: for $b \in \{0,1\}$, set $k_b := (k_b^<, [\![\mathbf{r}^{(\mathbf{q})}]\!]_b, [\![\mathbf{S}]\!]_b, [\![\mathbf{T}]\!]_b)$

$\mathsf{Eval}^{\mathsf{spline}}_{n,m,d,\mathbf{p},\mathbf{h}}(k_b, \hat{x})$: ▷ $\hat{x} \in \mathbb{Z}_N$

1: Parse k_b as $(k_b^<, [\![\mathbf{r}^{(\mathbf{q})}]\!]_b, [\![\mathbf{S}]\!]_b, [\![\mathbf{T}]\!]_b)$
2: **for** $i \in [m]$
3: $\quad h'_i := \hat{x} - h_i \bmod N$
4: $\quad [\![t_i]\!]_b := \mathsf{SPDZ\text{-}Eval}^<_{n,n}(k_b^<, h'_i)$
5: $h'_m := h'_0$
6: $[\![t_m]\!]_b := [\![t_0]\!]_b$
7: $[\![\hat{\mathbf{q}}]\!]_b := \sum_{i=0}^{m-1} ([\![t_{i+1}]\!]_b - [\![t_i]\!]_b + \mathbb{1}\{\alpha_i \le \alpha_{i+1}\}) \cdot \mathbf{p}_i + [\![\mathbf{r}^{(\mathbf{q})}]\!]_b$
8: $\hat{\mathbf{q}} \leftarrow \mathsf{auth\text{-}reconstruct}([\![\hat{\mathbf{q}}]\!]_b)$
9: **return** $\sum_{k=0}^{d} \left([\![\mathbf{T}]\!]_b[k] \cdot \hat{x}^k + \sum_{j=0}^{k} \binom{k}{j} [\![\mathbf{S}]\!]_b[k-j] \cdot \hat{\mathbf{q}}[k] \cdot \hat{x}^j \right)$

Figure 5: IFSS scheme for spline.

Our scheme has $\mathsf{keysize}(\mathsf{SPDZ\text{-}DCF}_{n,n}) + 6(d+1)(n+s)$ bits of preprocessing material; this is *independent* of m. In the online phase, the protocol communicates $(d+2)(n+s)$ bits and performs m evaluations of $\mathsf{SPDZ\text{-}Eval}^<_{n,n}$.

Comparison to prior work. We compare our scheme with prior work for evaluating a spline from Llama [35] for the sigmoid function with $m = 19$ and $d = 2$. For $n = s = 64$, our protocol requires $27,262$ bits of preprocessing material; the parties each send 512 bits and make 4864 calls to AES.

The spline protocol by Boyle et al. [6] requires preprocessing material that grows linearly with m. For the same parameters as above, their protocol requires $1,928,318$ bits of preprocessing material, which is more than $70\times$ larger than in our protocol. In the online phase, parties make $282,112$ AES calls ($58\times$ more than in our protocol), and each send 128 bits.

A spline protocol can be constructed in the framework of Escudero et al. [29] using m comparisons and $2d-1$ multiplications. Setting $\kappa = 40$ and otherwse using the same paramaters as above, thus would require at least $338,433$ bits of preprocessing material, which is $12\times$ larger than in our protocol, and require per-party communication of $7,703$, which is $15\times$ larger than in our protocol.

5.5. Reciprocal

For n-bit signed fixed-point numbers with precision f, the reciprocal of x is $\mathsf{Reciprocal}_{n,f}(x) = \mathsf{sign}(x) \cdot \lfloor 2^{2f}/|x| \rfloor$. (We assume the underlying circuit enforces $x \neq 0$.) Rather than giving an (I)FSS scheme for computing (the masked version of) this function, we describe how to compute the reciprocal using other gates we already know how to handle. In other words, we show how to replace reciprocal gates in the underlying circuit with other gates.

Our approach for handling reciprocal is inspired by the approach for computing $1/\sqrt{x}$ in Sigma [34]. That work first converts the input x into an appropriate floating-point representation[3] (d, m, e), representing the sign bit, the ℓ_m-bit mantissa, and the ℓ_e-bit exponent, respectively, so that $x \approx (-1)^d \cdot 2^e \cdot (1 + m/2^{\ell_m})$. A look-up table is then used to compute the output. We adopt this strategy along with a few optimizations.

First, we compute $\mathsf{sign}(x)$ and $|x|$, then compute the reciprocal of $|x|$ and adjust for sign later. This reduces the size of the needed look-up table in half. Second, since $\mathsf{Reciprocal}_{n,f}(x) = 0$ for $|x| > 2^{2f}$, the value $|x|$ can be clipped to lie in $(0, 2^{2f}]$. This allows us to operate on a value that can be represented using only $2f$ bits. We further reduce the size of the look-up table by a factor of 2^{ℓ_e} by observing that it suffices to do a look-up based on m alone and adjust for e later. With these optimizations, we only require a look-up table of size 128 when $n = 64$. We provide a detailed description of our protocol in Appendix B.3.

6. Evaluation

We implement our protocols as part of SHARK, a system for actively secure two-party machine-learning inference. We then empirically evaluate SHARK and compare its performance to prior work. We show that our protocols for various non-linear functions are 12–20$\times$ faster and use 4.8–33$\times$ less communication than the corresponding protocols of Escudero et al. [29] implemented in MP-SPDZ [40]. On end-to-end inference of feed-forward neural networks (FFNN) or convolutional neural networks (CNNs) over a LAN, SHARK outperforms the work of Escudero et al. [29] by 4.2–320$\times$ in communication and 11–115$\times$ in runtime. In a WAN, the speedups are even larger (31–2352$\times$). We also show that, even compared to a secure-inference protocol that provides weaker guarantees (i.e., one-sided security against a malicious client holding the input to the model), SHARK requires 8–16$\times$ less communication.

Finally, we note that SHARK enables the first actively secure transformer inference; prior tools for secure transformer inference all provide only semi-honest security. SHARK provides active security with less than $3\times$ overhead compared to the state-of-the-art semi-honest secure transformer-inference tool SIGMA [34].

Implementation. We implement SHARK in C++. using OpenMP [54] for multithreading and Eigen [33] for efficient matrix operations. We use a PRG from cryptoTools [59] that internally uses AES x86 instructions. Our implementation is available at https://github.com/kanav99/shark.

Evaluation. We perform all experiments on two machines, each using 4 threads, with AMD Epyc 7742 processors and 1 TB of RAM. We consider two network settings: (1) a LAN with 9.4 Gbps bandwidth and 0.05 ms round-trip latency and (2) a WAN with 320 Mbps bandwidth and 60 ms round-trip latency simulated using the `tc` command.

ML models. We evaluate protocols on models that have been considered previously in the literature on secure inference. Specifically, we consider an FFNN D4 (Benchmark 4 of DeepSecure [61]) for smart-sensing applications [28]. For the MNIST-10 dataset with 28×28 monochrome images, we consider the 2-layer CNN M1 from MiniONN [48]. For the CIFAR-10 dataset with 32×32 RGB images, we consider the 7-layer CNN M2 [48] and the 3-layer CNN HiNet [24]. For the ImageNet-1000 dataset [25] with 224×224 RGB images, we consider the 8-layer CNN AlexNet and the 16-layer CNN VGG16. For transformers, we evaluate BERT, the 110-million parameter BERT-Base model from MPC-Former [45], that approximates GeLU with a quadratic polynomial and exponentials in softmax with ReLUs.

Concrete parameters. All our benchmarks use $n = s = 64$ and $\kappa = 40$. For fixed-point numbers, we follow MP-SPDZ and set the precision to 16. For computing reciprocal, we use $f_{\mathsf{int}} = 20$ (cf. Appendix B.3).

6.1. Microbenchmarks

Table 2 compares the performance of our protocols for ReLU, ReluARS, spline, and reciprocal against the corresponding protocols of Escudero et al. [29] implemented in MP-SPDZ [40]. Results for ReLU and ReluARS are

3. This exploits the fact that *compact* floating-point representations of x (i.e., floating-point representations where $\ell_m + \ell_e + 1 \ll n$) are sufficiently accurate for most machine-learning applications.

averages taken over one million executions. Since Escudero et al. do not provide a dedicated protocol for ReluARS, we implement it for their work via sequential calls to ReLU followed by ARS (with $f = 16$). The spline microbenchmark is for a degree-2 spline with $m = 19$ intervals for sigmoid [35], averaged over 100,000 executions. We remark that for reciprocal, MP-SPDZ implements an approximation [14] based on Goldschmidt iterations that has roughly $2\times$ higher relative numerical error than SHARK.

For ReLU, the improvement in communication matches what is expected from analytical estimates. For ReluARS, the empirical communication improvements are slightly lower than our analytical estimates because our implementation communicates in increments of 1 byte, thus introducing some overhead due to padding. Apart from reciprocal, where the improvements are less significant in part because we perform a more-precise computation, SHARK is an order of magnitude faster than the work of Escudero et al. in both the LAN and WAN settings.

6.2. End-to-End Benchmarks

Table 3 compares the performance of SHARK and the work of Escudero et al. (implemented in MP-SPDZ) for end-to-end model inference. Our results show that SHARK outperforms prior work by 4–320$\times$ in communication and 11–2352$\times$ in runtime.

We also report the size of the preprocessing material and the round complexity of SHARK in Table 4; unfortunately, we are not able to compare these to the protocol of Escudero et al. since MP-SPDZ does not report those metrics reliably. Nevertheless, our estimates (cf. Table 1) indicate that they are at least an order of magnitude better in SHARK.

Comparison to SIMC++. The SIMC++ protocol [15] considers a weaker threat model where the client can be malicious but the model owner is assumed to be semi-honest. Since the code of SIMC++ is unavailable, we can only compare it to SHARK in terms of communication. Based on the reported nubmbers for SIMC++ [15, Table 4], we find that SHARK has $8\times$ lower communication for evaluating M1 and $16\times$ lower communication for evaluating M2, even though it considers a stronger threat model.

Comparison to MD-ML. As discussed in Section 5.2, MD-ML [70] uses local truncations that are insecure. Nevertheless, we provide a brief comparison to SHARK. MD-ML's public implementation [50] only supports AlexNet. On this benchmark, MD-ML has $1.6\times$ higher communication than SHARK while being $2.65\times$ slower in the LAN setting.

Transformer inference. Two prior works show passively secure protocols for transformer inference. MPCFormer [45] uses knowledge distillation to generate BERT models that approximate expensive operations (e.g., GeLU, softmax, etc.) with operations that are easier to compute (e.g., ReLU, quadratics, reciprocal), and then evaluates the resulting models using CrypTen [42]. SIGMA [34] represents the state-of-

Function		Comm. (MB)	Runtime (seconds)	
			LAN	WAN
ReLU	SHARK	32.42	0.56	1.45
	[29]	155.26 (4.78×)	10.27 (18.43×)	17.60 (12.15×)
ReluARS	SHARK	36.24	1.16	2.15
	[29]	275.77 (7.60×)	17.62 (15.14×)	28.41 (13.19×)
Spline	SHARK	15.64	1.08	1.75
	[29]	521.91 (33.33×)	20.81 (19.32×)	35.32 (20.12×)
Reciprocal	SHARK	221.25	12.92	18.65
	[29]	1282.29 (5.79×)	24.78 (1.91×)	58.10 (3.11×)

TABLE 2: Microbenchmarks.

Benchmark		Comm. (MB)	Runtime (seconds)	
			LAN	WAN
D4	SHARK	187.74	0.32	6.50
	[29]	794.49 (4.23×)	5.29 (16×)	203.66 (31×)
M1	SHARK	1.19	0.03	1.61
	[29]	45.31 (38.07×)	1.12 (41×)	361.71 (224×)
M2	SHARK	14.51	0.40	2.85
	[29]	3712.05 (255.82×)	7.39 (18×)	548.54 (192×)
HiNet	SHARK	9.52	0.17	2.62
	[29]	1231.13 (129.32×)	20.01 (115×)	6163.01 (2352×)
AlexNet	SHARK	2040.55	12.61	67.93
	[29]	71234.20 (34.90×)	144.66 (11×)	6178.56 (90×)
VGG16	SHARK	3000.91	66.70	144.22
	[29]	960828.00 (320.17×)	1839.68 (27×)	240636.00 (1668×)

TABLE 3: Neural-network inference benchmarks. For tanh activations in D4, SHARK uses a spline approximation [35] while the protocol of Escudero et al. offers native support.

Benchmark	Rounds	Preprocessing material (MB)
D4	25	577
M1	28	21
M2	36	402
HiNet	44	233
AlexNet	38	7,460
VGG16	76	34,852

TABLE 4: Number of rounds and size of the preprocessing material for SHARK.

the-art for semi-honest secure inference of BERT models without such approximations.

We compare to SHARK using the same model architecture from MPCFormer.[4] For BERT-Base, SHARK requires 41 seconds (over a LAN) and communicates 2.7 GB to perform inference on an 128-token input. This is an improvement of $1.04\times$ in communication and $3\times$ in runtime as compared to MPCFormer with CrypTen, even though SHARK targets stronger security. On the other hand, compared to SIGMA we find that SHARK is roughly $2\times$ slower and communicates roughly $3\times$ more. These are reasonable overheads considering the stronger threat model supported by SHARK.

References

[1] Nitin Agrawal, Ali Shahin Shamsabadi, Matt J. Kusner, and Adrià Gascón. QUOTIENT: Two-party secure neural network training and prediction. In *CCS*, 2019.

[2] Y. Akimoto, K. Fukuchi, Y. Akimoto, and J. Sakuma. Privformer: Privacy-preserving transformer with MPC. In *EuroS&P*, 2023.

[3] Donald Beaver. Efficient multiparty protocols using circuit randomization. In *Advances in Cryptology—Crypto '91*, 1991.

[4] Rikke Bendlin, Ivan Damgård, Claudio Orlandi, and Sarah Zakarias. Semi-homomorphic encryption and multiparty computation. In *Eurocrypt*, 2011.

[5] Fabian Boemer, Rosario Cammarota, Daniel Demmler, Thomas Schneider, and Hossein Yalame. MP2ML: A mixed-protocol machine learning framework for private inference. In *ARES*, 2020.

[6] Elette Boyle, Nishanth Chandran, Niv Gilboa, Divya Gupta, Yuval Ishai, Nishant Kumar, and Mayank Rathee. Function secret sharing for mixed-mode and fixed-point secure computation. In *Eurocrypt*, 2020.

[7] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing. In *Eurocrypt*, 2015.

[8] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In *CCS*, 2016.

[9] Elette Boyle, Niv Gilboa, and Yuval Ishai. Secure computation with preprocessing via function secret sharing. In *TCC*, 2019.

[10] Andreas Brüggemann, Oliver Schick, Thomas Schneider, Ajith Suresh, and Hossein Yalame. Don't eject the impostor: Fast three-party computation with a known cheaters. In *IEEE S&P*, 2023.

[11] Niklas Büscher, Daniel Demmler, Stefan Katzenbeisser, David Kretzmer, and Thomas Schneider. HyCC: Compilation of Hybrid Protocols for Practical Secure Computation. In *CCS*, 2018.

[12] Megha Byali, Harsh Chaudhari, Arpita Patra, and Ajith Suresh. FLASH: Fast and robust framework for privacy-preserving machine learning. *PETS*, 2020(2):459–480, 2020.

[13] Ran Canetti. Security and composition of multiparty cryptographic protocols. *J. Cryptology*, 2000.

[14] Octavian Catrina and Amitabh Saxena. Secure computation with fixed-point numbers. In *Financial Cryptography and Data Security*, 2010.

[15] Nishanth Chandran, Divya Gupta, Sai Lakshmi Bhavana Obbattu, and Akash Shah. SIMC: Ml inference secure against malicious clients at semi-honest cost. In *USENIX Security Symposium*, 2022.

[16] Nishanth Chandran, Divya Gupta, Aseem Rastogi, Rahul Sharma, and Shardul Tripathi. EzPC: Programmable and efficient secure two-party computation for machine learning. In *EuroS&P*. IEEE, 2019.

4. https://github.com/DachengLi1/MPCFormer/

[17] Harsh Chaudhari, Arpita Patra, Rahul Rachuri, and Ajith Suresh. Tetrad: Actively secure 4PC for secure training and inference. In *NDSS*, 2022.

[18] Hao Chen, Miran Kim, Ilya Razenshteyn, Dragos Rotaru, Yongsoo Song, and Sameer Wagh. Maliciously secure matrix multiplication with applications to private deep learning. In *Asiacrypt*, 2020.

[19] Ronald Cramer, Ivan Damgård, Daniel Escudero, Peter Scholl, and Chaoping Xing. SPDZ2k: Efficient MPC mod 2^k for dishonest majority. In *Crypto*, 2018.

[20] Anders Dalskov, Daniel Escudero, and Marcel Keller. Fantastic four: Honest-majority four-party secure computation with malicious security. In *USENIX Security*, 2021.

[21] Anders P. K. Dalskov, Daniel Escudero, and Marcel Keller. Secure evaluation of quantized neural networks. *PoPETs*, 2020.

[22] Ivan Damgård, Daniel Escudero, Tore Kasper Frederiksen, Marcel Keller, Peter Scholl, and Nikolaj Volgushev. New primitives for actively-secure MPC over rings with applications to private machine learning. In *IEEE S&P*. IEEE, 2019.

[23] Ivan Damgård, Valerio Pastro, Nigel Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In *Crypto*, 2012.

[24] Deep Learning Training with Multi-Party Computation. https://github.com/csiro-mlai/deep-mpc/tree/more-models.

[25] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *CVPR*, 2009.

[26] Caiqin Dong, Jian Weng, Jia-Nan Liu, Yue Zhang, Yao Tong, Anjia Yang, Yudan Cheng, and Shun Hu. Fusion: Efficient and secure inference resilient to malicious servers. In *NDSS*, 2023.

[27] Ye Dong, Wen jie Lu, Yancheng Zheng, Haoqi Wu, Derun Zhao, Jin Tan, Zhicong Huang, Cheng Hong, Tao Wei, and Wenguang Chen. Puma: Secure inference of llama-7b in five minutes, 2023.

[28] UCI machine learning repository. https://archive.ics.uci.edu/ml/datasets/Daily+and+Sports+Activities.

[29] Daniel Escudero, Satrajit Ghosh, Marcel Keller, Rahul Rachuri, and Peter Scholl. Improved primitives for MPC over mixed arithmetic-binary circuits. In *Crypto*, 2020.

[30] Tore Kasper Frederiksen, Marcel Keller, Emmanuela Orsini, and Peter Scholl. A unified approach to MPC with preprocessing using OT. In *Asiacrypt*, 2015.

[31] Ran Gilad-Bachrach, Nathan Dowlin, Kim Laine, Kristin E. Lauter, Michael Naehrig, and John Wernsing. CryptoNets: Applying neural networks to encrypted data with high throughput and accuracy. In *ICML*, 2016.

[32] Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game or a completeness theorem for protocols with honest majority. In *STOC*, 1987.

[33] Gaël Guennebaud and Benoît Jacob. Eigen v3. http://eigen.tuxfamily.org, 2010.

[34] Kanav Gupta, Neha Jawalkar, Ananta Mukherjee, Nishanth Chandran, Divya Gupta, Ashish Panwar, and Rahul Sharma. SIGMA: Secure GPT inference with function secret sharing. In *PETS*, 2024.

[35] Kanav Gupta, Deepak Kumaraswamy, Nishanth Chandran, and Divya Gupta. Llama: A low latency math library for secure inference. In *PETS*, 2022.

[36] Meng Hao, Hongwei Li, Hanxiao Chen, Pengzhi Xing, Guowen Xu, and Tianwei Zhang. Iron: Private inference on transformers. In *NeurIPS*, 2022.

[37] Zhicong Huang, Wen jie Lu, Cheng Hong, and Jiansheng Ding. Cheetah: Lean and fast secure two-party deep neural network inference. In *USENIX Security Symposium*, 2022.

[38] Neha Jawalkar, Kanav Gupta, Arkaprava Basu, Nishanth Chandran, Divya Gupta, and Rahul Sharma. Orca: FSS-based secure training and inference with GPUs. In *IEEE S&P*, 2024.

[39] Chiraag Juvekar, Vinod Vaikuntanathan, and Anantha Chandrakasan. Gazelle: A low latency framework for secure neural network inference. In *USENIX Security Symposium*, 2018.

[40] Marcel Keller. Multi-protocol SPDZ: Versatile framework for multi-party computation, 2019. https://github.com/data61/MP-SPDZ.

[41] Marcel Keller and Ke Sun. Secure quantized training for deep learning. In *ICML*, 2022.

[42] Brian Knott, Shobha Venkataraman, Awni Hannun, Shubhabrata Sengupta, Mark Ibrahim, and Laurens van der Maaten. CrypTen: Secure multi-party computation meets machine learning. In *NeurIPS*, 2021.

[43] Nishat Koti, Mahak Pancholi, Arpita Patra, and Ajith Suresh. SWIFT: Super-fast and robust privacy-preserving machine learning. In *USENIX Security Symposium*, 2021.

[44] Ryan Lehmkuhl, Pratyush Mishra, Akshayaram Srinivasan, and Raluca Ada Popa. Muse: Secure inference resilient to malicious clients. In *USENIX Security Symposium*, 2021.

[45] Dacheng Li, Hongyi Wang, Rulin Shao, Han Guo, Eric Xing, and Hao Zhang. MPCFormer: Fast, performant and private transformer inference with MPC. In *ICLR*, 2023.

[46] Yun Li, Yufei Duan, Zhicong Huang, Cheng Hong, Chao Zhang, and Yifan Song. Efficient 3PC for binary circuits with application to maliciously-secure DNN inference. In *USENIX Security Symposium*, 2023.

[47] Yehuda Lindell and Benny Pinkas. Privacy preserving data mining. *J. Cryptol.*, 15(3):177–206, 2002.

[48] Jian Liu, Mika Juuti, Yao Lu, and N. Asokan. Oblivious neural network predictions via miniONN transformations. In *CCS*, 2017.

[49] Zoltán Ádám Mann, Christian Weinert, Daphnee Chabal, and Joppe W. Bos. Towards practical secure neural network inference: The journey so far and the road ahead. *ACM Comput. Surv.*, 56(5):117:1–117:37, 2024.

[50] MD-ML. https://github.com/NemoYuan2008/MD-ML.

[51] Pratyush Mishra, Ryan Lehmkuhl, Akshayaram Srinivasan, Wenting Zheng, and Raluca Ada Popa. Delphi: A cryptographic inference service for neural networks. In *USENIX Security Symposium*, 2020.

[52] Payman Mohassel and Peter Rindal. ABY^3: A mixed protocol framework for machine learning. In *CCS*, 2018.

[53] Payman Mohassel and Yupeng Zhang. SecureML: A system for scalable privacy-preserving machine learning. In *IEEE S&P*, 2017.

[54] OpenMP. https://www.openmp.org.

[55] Qi Pang, Jinhao Zhu, Helen Möllering, Wenting Zheng, and Thomas Schneider. BOLT: Privacy-preserving, accurate and efficient inference for transformers. In *IEEE S&P*, 2024.

[56] Arpita Patra and Ajith Suresh. Blaze: Blazing fast privacy-preserving machine learning. In *NDSS*, 2020.

[57] Deevashwer Rathee, Mayank Rathee, Rahul Kranti Kiran Goli, Divya Gupta, Rahul Sharma, Nishanth Chandran, and Aseem Rastogi. SIRNN: A math library for secure inference of RNNs. In *IEEE S&P*, 2021.

[58] Deevashwer Rathee, Mayank Rathee, Nishant Kumar, Nishanth Chandran, Divya Gupta, Aseem Rastogi, and Rahul Sharma. CrypTFlow2: Practical 2-party secure inference. In *CCS*, 2020.

[59] Peter Rindal. cryptoTools. https://github.com/ladnir/cryptoTools, 2023.

[60] Dragos Rotaru and Tim Wood. MArBled circuits: Mixing arithmetic and boolean circuits with active security. In *INDOCRYPT*, 2019.

[61] Bita Darvish Rouhani, M. Sadegh Riazi, and Farinaz Koushanfar. DeepSecure: Scalable provably-secure deep learning. In *DAC*, 2018.

[62] Théo Ryffel, David Pointcheval, and Francis Bach. ARIANN: Low-interaction privacy-preserving deep learning via function secret sharing. In *PETS*, 2022.

[63] Manuel B. Santos, Dimitris Mouris, Mehmet Ugurbil, Stanislaw Jarecki, José Reis, Shubho Sengupta, and Miguel de Vega. Curl: Private LLMs through wavelet-encoded look-up tables. In *CAMLIS*, 2024.

[64] Kyle Storrier, Adithya Vadapalli, Allan Lyons, and Ryan Henry. Grotto: Screaming fast $(2+1)$-PC for $\mathbb{Z}_{2^n}$ via (2, 2)-DPFs. In *ACM CCS*, 2023.

[65] Sameer Wagh. Pika: Secure computation using function secret sharing over rings. *PoPETs*, 2022.

[66] Sameer Wagh, Shruti Tople, Fabrice Benhamouda, Eyal Kushilevitz, Prateek Mittal, and Tal Rabin. Falcon: Honest-majority maliciously secure framework for private deep learning. *PoPETs*, 2021.

[67] Frank Wang, Catherine Yun, Shafi Goldwasser, Vinod Vaikuntanathan, and Matei Zaharia. Splinter: Practical private queries on public data. In *NSDI*, 2017.

[68] Jean-Luc Watson, Sameer Wagh, and Raluca Ada Popa. Piranha: A GPU platform for secure computation. In *USENIX Security Symposium*, 2022.

[69] Andrew Yao. How to generate and exchange secrets. In *FOCS*, 1986.

[70] Boshi Yuan, Shixuan Yang, Yongxiang Zhang, Ning Ding, Dawu Gu, and Shi-Feng Sun. MD-ML: Super fast privacy-preserving machine learning for malicious security with a dishonest majority. In *USENIX Security*, 2024.

[71] Jiawen Zhang, Xinpeng Yang, Lipeng He, Kejia Chen, Wen jie Lu, Yinghao Wang, Xiaoyang Hou, Jian Liu, Kui Ren, and Xiaohu Yang. Secure transformer inference made non-interactive. In *NDSS*, 2025.

Appendix A.
Security Proof for the Overall Protocol

Consider a circuit $\mathcal{M}$ with ℓ levels, where each level f_i takes as input some of the previously computed wire values and produces the corresponding output. Let x and y be the inputs to the circuit provided by P_0 and P_1, respectively. Recall that, by construction of our protocol (cf. Section 3.2), for each level f_i we have an (I)FSS scheme (Gen_i, Π_i), with auxiliary function AUX_i, that implements the function $\{f_i(x - \mathsf{r}_{\mathsf{in}}) + \tilde{\mathsf{r}}_{\mathsf{out}}\}_{\mathsf{r}_{\mathsf{in}}, \tilde{\mathsf{r}}_{\mathsf{out}}}$ along with the corresponding authentication tags. Note that in our protocol, AUX_i represents opened shares and their tags. By definition of (I)FSS, there is a simulator $\mathcal{S}_i$ that can simulate an adversary's view as described in Section 2.4. We now build a simulator $\mathcal{S}$ for the overall protocol.

Simulator. Without loss of generality, assume P_0 is corrupted by the adversary $\mathcal{A}$. The simulator $\mathcal{S}$ samples random keys $\Delta^{\mathsf{A}}, \Delta^{\mathsf{B}}$ and masks for all wires in the circuit, including input masks $\mathsf{r}^{(x)}, \mathsf{r}^{(y)}$ and output mask $\mathsf{r}^{(z)}$. It then sequentially invokes the simulator $\mathcal{S}_i$ for each level i to generate (I)FSS key $k_{i,0}$, and sends $\mathsf{r}^{(x)}, k_{0,0} \ldots, k_{\ell-1,0}, [\![-\mathsf{r}^{(z)}]\!]_0$ and random shares of $\Delta^{\mathsf{A}}, \Delta^{\mathsf{B}}$ to $\mathcal{A}$.

In the online phase, $\mathcal{S}$ sends $\mathsf{r}^{(y)}$ to $\mathcal{A}$. On recieving $\hat{x}$ from $\mathcal{A}$, the simulator sends $x := \hat{x} - \mathsf{r}^{(x)}$ to the ideal functionality and receives output z in return. To simulate the computation of the ith level, the simulator invokes $\mathcal{S}_i$ with the masked input generated in prior levels (at the initial level, the masked inputs are simply $\hat{x}$ and $\hat{y}$); as a result, it

observes the messages sent by $\mathcal{A}$ and generates an output share $[\![\hat{y}_i]\!]_0$ and auxiliary information $\mathsf{aux}_{i,0}$. As it knows Δ^{A} and Δ^{B}, it can calculate a share $[\![\hat{y}_i]\!]_1$ such that $([\![\hat{y}_i]\!]_0, [\![\hat{y}_i]\!]_1)$ form valid shares of a random value. It sends $[\![\hat{y}_i]\!]_1$ (without the authentiction tag) to $\mathcal{A}$, and receives a value $[\![\hat{y}_i{}']\!]_0$ (that may not equal $[\![\hat{y}_i]\!]_0$) in response; this defines the masked output of this level. Note that the simulator also knows the corresponding authentication tags for all values (including $[\![\hat{y}_i]\!]_1$) that it sent to $\mathcal{A}$ in this stage. Following simulation of all levels of the circuit, let $\hat{z}$ denote the masked output.

Next, the simulator performs the batch check with $\mathcal{A}$. If this check fails (as it will with overwhelming probability if $\mathcal{A}$ sent an incorrect value at some point during the protocol), the simulator sends abort to the ideal functionality. Otherwise, it computes $[\![z]\!]_0 := \hat{z} - [\![\mathsf{r}^{(z)}]\!]_0$. As it knows z (sent by the ideal functionality) and all global keys, it can generate $[\![z]\!]_1$ such that the pair of shares $([\![z]\!]_0, [\![z]\!]_1)$ are a valid sharing of z. Once again, it performs an authentication check with the adversary to recover the output; if the check fails, the simulator sends abort to the ideal functionality, and otherwise it sends continue so that the honest party in the ideal world also receives the output.

Hybrids. We prove the indistinguishability of the real-world and ideal-world view using a hybrid argument.

Hybrid $\mathcal{H}_0$. In this hybrid, we let the simulator $\mathcal{S}$ know the honest party's input. The simulator runs the protocol, taking roles of the dealer and honest party, according to the transcript with the adversary. Hence, this hybrid is identical to the real-world execution.

Hybrid $\mathcal{H}_i$ ($i \in \{1, 2 \dots \ell\}$). In these hybrids as well, the simulator knows the honest party's inputs. It follows the protocol for the input phase and the first $\ell - i$ levels. For level $j \in \{\ell - i \dots \ell - 1\}$ computing function f_j, we proceed as follows: it uses simulator $\mathcal{S}_j$ to first generate the key $k_{j,0}$ in the preprocessing phase. Then, in the online phase, it passes the masked value $\hat{x}$, generated by a prior level, to $\mathcal{S}_j$ to interact with $\mathcal{A}$ and outputs adversary's (honest) output share $[\![\hat{y}_j]\!]_0$ and $\mathsf{aux}_{j,0}$. Calculates $[\![\hat{y}_j]\!]_1$ s.t. the pair $([\![\hat{y}_j]\!]_0, [\![\hat{y}_j]\!]_1)$ opens to some random value. Calculates $\mathsf{aux}_{j,1} = \mathcal{S}_{\mathsf{AUX}}(b = 0, p = (\mathsf{r}_{\mathsf{in}}, \tilde{\mathsf{r}}_{\mathsf{out}}, \Delta^{\mathsf{A}}, \Delta^{\mathsf{B}}), \mathsf{aux}_{j,0})$. It uses $\mathsf{aux}_{j,1}$ to calculate z-values for all intermediate reconstructions and appends them to global batch check buffer buf_1. It opens $\hat{y_j}$ and uses it in the subsequent simulated levels. It performs the output phase, using buf_1, as per the protocol, and opens final output z with output share adjustment.

Hybrid $\mathcal{H}_{\ell+1}$. This hybrid is similar to $\mathcal{H}_\ell$, but additionally uses dummy inputs. Hence, it is identical to the ideal-world execution.

Indistinguishability. We prove indistinguishability of successive hybrids from each other.

For $i \in \{0, 1 \dots \ell - 1\}$, $\mathcal{H}_i$ and $\mathcal{H}_{i+1}$. Note that $\mathcal{H}_i$ and $\mathcal{H}_{i+1}$ differ only in the jth level, where $j = \ell - i - 1$. For prior levels, both hybrids run the actual protocol. For subsequent levels, both hybrids run simulators. Let's assume for the contradiction that there is an adversary $\mathcal{D}$ that can distinguish between $\mathcal{H}_i$ and $\mathcal{H}_{i+1}$ for some inputs x' and y'. We show that using $\mathcal{D}$, we can devise an adversary $\mathcal{D}^*$ that contradicts security of the (I)FSS scheme for f_j.

We now describe $\mathcal{D}^*$. $\mathcal{D}^*$ internally runs the protocol, according to $\mathcal{D}$, until the $(j-1)$th level, with inputs x' and y'. It learns global keys $\Delta^{\mathsf{A}}, \Delta^{\mathsf{B}}$. It also learns the intermediate masks of all the other levels. Let $\mathsf{r}^{(x)}$ and $\tilde{\mathsf{r}}^{(y)}$ denote the masks of the input and output of jth level. It sends $\mathsf{r}^{(x)}, \tilde{\mathsf{r}}^{(y)}, \Delta^{\mathsf{A}}, \Delta^{\mathsf{B}}$ to the challenger and receives k_0 in return. It then sends $\hat{x}$ to the challenger and runs Π_j, as directed by $\mathcal{D}$, with the challenger. It internally runs the rest of the protocol as per $\mathcal{D}$ which outputs b as the prediction bit. Note that if $b = 0$, $\mathcal{D}$ sees a view identical to $\mathcal{H}_i$. If $b = 1$, it sees a view identical to the one in $\mathcal{H}_{i+1}$. As $\mathcal{D}$ can distinguish between the two, $\mathcal{D}^*$ can distinguish between the real and simulated views, as per (I)FSS definition.

$\mathcal{H}_\ell$ and $\mathcal{H}_{\ell+1}$. In both hybrids, the input (dummy or real) is masked with a uniformly sampled value, hence the distribution of masked values $\hat{x}, \hat{y}$ is identical. As the rest of the computation is the same PPT over these indistinguishable masked inputs, the entire view generated in both cases are indistinguishable.

Appendix B. Additional FSS Schemes

B.1. Matrix Multiplication

Let $\hat{X}, \mathsf{r}^{(1)}_{\mathsf{in}} \in \mathbb{Z}_N^{d_0 \times d_1}$, let $\hat{Y}, \mathsf{r}^{(2)}_{\mathsf{in}} \in \mathbb{Z}_N^{d_1 \times d_2}$, and let $\tilde{\mathsf{r}}_{\mathsf{out}} \in \mathbb{Z}_{\tilde{N}}^{d_0 \times d_2}$. Ignoring authentication, we have

$$\begin{aligned}
&\mathsf{MatMul}_{\mathsf{r}^{(1)}_{\mathsf{in}}, \mathsf{r}^{(2)}_{\mathsf{in}}, \tilde{\mathsf{r}}_{\mathsf{out}}}(\hat{X}, \hat{Y}) \\
&\quad = \quad (\hat{X} - \mathsf{r}^{(1)}_{\mathsf{in}}) \cdot (\hat{Y} - \mathsf{r}^{(2)}_{\mathsf{in}}) + \tilde{\mathsf{r}}_{\mathsf{out}} \\
&\quad = \quad \hat{X} \cdot \hat{Y} - \mathsf{r}^{(1)}_{\mathsf{in}} \cdot \hat{Y} - \hat{X} \cdot \mathsf{r}^{(2)}_{\mathsf{in}} + \mathsf{r}^{(1)}_{\mathsf{in}} \cdot \mathsf{r}^{(2)}_{\mathsf{in}} + \tilde{\mathsf{r}}_{\mathsf{out}}.
\end{aligned}$$

Since $\hat{X}, \hat{Y}$ are known to both parties, this is a linear combination of $\mathsf{r}^{(1)}_{\mathsf{in}}$, $\mathsf{r}^{(2)}_{\mathsf{in}}$, and $\mathsf{r}^{(1)}_{\mathsf{in}} \cdot \mathsf{r}^{(2)}_{\mathsf{in}} + \tilde{\mathsf{r}}_{\mathsf{out}}$. Thus, we can set the keys for the parties to be $\mathsf{SPD}\mathbb{Z}_{2^k}$-shares of those values; the parties can compute their output by performing homomorphic operations over those shares. The resulting scheme (with authentication included) is in Figure 6.

$\mathsf{Gen}^{\mathsf{MatMul}}_{n,d_0,d_1,d_2}(\mathsf{r}^{(1)}_{\mathsf{in}}, \mathsf{r}^{(2)}_{\mathsf{in}}, \tilde{\mathsf{r}}_{\mathsf{out}}, \Delta^{\mathsf{A}})$:

1: $u := \mathsf{r}^{(1)}_{\mathsf{in}}, v := \mathsf{r}^{(2)}_{\mathsf{in}}, \; w := \mathsf{r}^{(1)}_{\mathsf{in}} \cdot \mathsf{r}^{(2)}_{\mathsf{in}} + \tilde{\mathsf{r}}_{\mathsf{out}}$
2: $[\![u]\!] \leftarrow \mathsf{auth\text{-}ashare}(u)$, $[\![v]\!] \leftarrow \mathsf{auth\text{-}ashare}(v)$
3: $[\![w]\!] \leftarrow \mathsf{auth\text{-}ashare}(w)$
4: for $b \in \{0, 1\}$, set $k_b := ([\![u]\!]_b, [\![v]\!]_b, [\![w]\!]_b)$

$\mathsf{Eval}^{\mathsf{MatMul}}_{n,d_0,d_1,d_2}(k_b, (\hat{X}, \hat{Y}))$:

1: Parse k_b as $([\![u]\!]_b, [\![v]\!]_b, [\![w]\!]_b)$
2: Output $[\![\hat{Z}]\!]_b := \hat{X} \cdot \hat{Y} - [\![u]\!]_b \cdot \hat{Y} - \hat{X} \cdot [\![v]\!]_b + [\![w]\!]_b$

Figure 6: FSS scheme for matrix multiplication.

B.2. Look-up Tables

Let $\mathbf{T} \in \mathbb{Z}_L^N$. Observe that $\mathbf{T}[\hat{x} - \mathsf{r}_{\mathsf{in}}]$ is equal to the dot product of $\mathbf{T}$ with the one-hot vector having a single 1 in position $\hat{x} - \mathsf{r}_{\mathsf{in}}$. Thus, an FSS scheme for look-ups using a masked index can be constructed from a DPF. (An output mask and authentication can also be incorporated easily.) See Figure 7. The scheme requires preprocessing of size $\mathsf{keysize}(\mathsf{SPDZ\text{-}DPF}_{n,\ell}) + 2(\ell + s)$ bits. We remark that $\mathbf{T}$ need not be known at the time the keys are generated.

$\mathsf{Gen}^{\mathsf{LUT}}_{n,\ell,\mathbf{T}}(\mathsf{r}_{\mathsf{in}}, \tilde{\mathsf{r}}_{\mathsf{out}}, \Delta^{\mathsf{A}})$: $\triangleright$ $\mathsf{r}_{\mathsf{in}} \in \mathbb{Z}_N, \tilde{\mathsf{r}}_{\mathsf{out}} \in \mathbb{Z}_{\widetilde{L}}$

1: $(k_0^\bullet, k_1^\bullet) \leftarrow \mathsf{SPDZ\text{-}Gen}^\bullet_{n,\ell}(-\mathsf{r}_{\mathsf{in}} \bmod N, 1)$
2: $[\![\mathsf{r}_{\mathsf{out}}]\!] \leftarrow \mathsf{auth\text{-}ashare}(\tilde{\mathsf{r}}_{\mathsf{out}})$
3: for $b \in \{0,1\}$, set $k_b := (k_b^\bullet, [\![\mathsf{r}_{\mathsf{out}}]\!]_b)$

$\mathsf{Eval}^{\mathsf{LUT}}_{n,\ell,\mathbf{T}}(k_b, \hat{x})$: $\triangleright$ $\hat{x} \in \mathbb{Z}_N$

1: Parse k_b as $(k_b^\bullet, [\![\mathsf{r}_{\mathsf{out}}]\!]_b)$
2: $[\![\mathbf{u}]\!]_b \leftarrow \mathsf{SPDZ\text{-}EvalAll}^\bullet_{n,\ell}(k_b^\bullet)$
3: Output $[\![\hat{y}]\!]_b := [\![\mathsf{r}_{\mathsf{out}}]\!]_b + \sum_{i=0}^{N-1} [\![\mathbf{u}[i]]\!]_b \cdot \mathbf{T}[\hat{x} + i]$

Figure 7: FSS scheme for look-up tables.

B.3. Reciprocal

We deviate from our usual approach of describing an (I)FSS scheme for (the masked and authenticated version of) the function of interest. Instead, we describe how to compute (an approximation to) reciprocal based on other functions. Reciprocal gates in the overall circuit can then be replaced by gates for those other functions, for which we already have corresponding (I)FSS schemes.

To compute the reciprocal of an input x, we start by first computing $d = \mathsf{MSB}(x)$ and $a = |x|$. The first can be done using a comparison (cf. Eq. (1)), and the second can then be computed as $|x| = x - 2 \cdot \mathsf{select}(d, x)$. We show next how to compute $z' = \mathsf{Reciprocal}_{n,f}(a)$, and adjust the sign of the result at the end based on d.

Let $F = 2^{2f}$. Since $a > 0$ and $\mathsf{Reciprocal}_{n,f}(a) = 0$ when $a > F$, the desired result is non-zero only for 2^{2f} values $a \in \{1, 2, \ldots, 2^{2f}\}$. Hence, we can compute the desired result using $(a-1) \bmod F$, outputting 0 if $a > F$. Concretely, if we let

$$c = \mathbb{1}\{a \le F\} = \mathbb{1}\{a < F + 1\} = \mathsf{GEZ}_n(a - F - 1) \oplus 1$$

and $y = (a-1) \bmod F$, then we can compute $z = \mathsf{Reciprocal}'_{2f,f}(y) = \mathsf{Reciprocal}_{2f,f}(y+1)$ and select between that result and 0 at the end based on c.

To compute $\mathsf{Reciprocal}'_{2f,f}(y)$, we use a two-step approach similar to the one used in Sigma [34] to calculate inverse square root: convert the input to a floating-point representation, then use a look-up table. Let $m \in \mathbb{Z}_{2^{\ell_m}}$ and $e \in \mathbb{Z}_{2^{\ell_e}}$ represent the ℓ_m-bit mantissa and ℓ_e-bit exponent, respectively, in the floating-point representation of $y+1$. (As $y+1$ is positive, its sign bit is 0.) So, $y+1 \approx (1 + m/2^{\ell_m}) \cdot 2^e$, by definition of floating-point representation. Therefore,

$$\begin{aligned}\lfloor 2^{2f}/(y+1) \rfloor &\approx \lfloor 2^{2f-e}/(1 + m/2^{\ell_m}) \rfloor \\ &= \lfloor 2^{2f-e+\ell_m} \cdot 1/(2^{\ell_m} + m) \rfloor \\ &= \lfloor \left(2^{2f-e+\ell_m}\right) \cdot \left(2^{f_{\mathsf{int}}}/(2^{\ell_m} + m)\right) \cdot 2^{-f_{\mathsf{int}}} \rfloor\end{aligned}$$

for some intermediate precision f_{int}. We can compute the desired output by separately calculating $t_1 = 2^{2f-e+\ell_m}$ and $t_2 = \lfloor 2^{f_{\mathsf{int}}}/(2^{\ell_m} + m) \rfloor$, multiplying those results, and then applying a logical right shift by f_{int} positions.

Calculating t_1. Observe that t_1 depends directly on e, which is the largest value for which 2^e is smaller than $y+1$. We can thus compute t_1 using a degree-0 spline by setting $\mathbf{H} = (0, 1, 3, 7, \ldots 2^{2f} - 1, 2^{2f})$ and $\mathbf{P}$ to degree-0 (i.e., constant) polynomials with $\mathbf{p}_i = 2^{2f-i+\ell_m}$ for $i \in [2f+1]$.

Calculating t_2. To calculate t_2, we follow an approach similar to that used in prior work [34], [38]. We first compute $u = 2^{n-e-1} = 2^{n-1-2f-\ell_m} \cdot t_1$. Multiplying this value with $a = y + 1$ results in a value w such that the ℓ_m-bits following the MSB in w are the mantissa m. So, $m = \mathsf{LRS}_{n,n-\ell_m-1}(u \cdot a) \bmod 2^{\ell_m}$. Given m, we now calculate $t_1 = 2^{f_{\mathsf{int}}}/(2^{\ell_m} + m)$ using a ℓ_m-bit LUT by invoking the protocol $\Pi^{\mathsf{LUT}}_{\ell_m,n,\mathbf{T}}$, where $\mathbf{T}$ is a table satisfying $\mathbf{T}[i] = 2^{f_{\mathsf{int}}}/(2^{\ell_m} + i)$ for all $i \in \mathbb{Z}_{2^{\ell_m}}$.

Based on the above discussion, we provide the algorithm for computing reciprocal in Figure 8.

Algorithm $\mathsf{Reciprocal}_{n,f}(x)$

1: $d := \mathsf{GEZ}(x) \oplus 1$
2: $a := x - 2 \cdot \mathsf{select}(d, x)$
3: $c := \mathsf{GEZ}(a - F - 1) \oplus 1$
4: $y := (a-1) \bmod F$
5: $t_1 := \mathsf{spline}_{2f,2f+1,0,\mathbf{P},\mathbf{H}}(y)$
6: $u := 2^{n-1-2f-\ell_m} \cdot t_1$
7: $w := u \cdot a$
8: $m := \mathsf{LRS}_{n,n-\ell_m-1}(w)$
9: $t_2 := \mathbf{T}[m]$
10: $h := t_1 \cdot t_2$
11: $z := \mathsf{LRS}_{n,f_{\mathsf{int}}}(h)$
12: $z' := \mathsf{select}(c, z)$
13: **return** $z' - 2 \cdot \mathsf{select}(d, z')$

Figure 8: Algorithm for computing reciprocal. $\mathbf{T}$ is a table of size 2^{ℓ_m} with $\mathbf{T}[i] = 2^{f_{\mathsf{int}}}/(2^{\ell_m} + i)$ for $i \in [2^{\ell_m}]$. $\mathbf{H}$ and $\mathbf{P}$ define a $(2f+1, 0)$-spline where $\mathbf{H} = (0, 1, 3, \ldots, 2^{2f} - 1, 2^{2f})$ and $\mathbf{P} = \{\mathbf{p}_0, \mathbf{p}_1, \ldots \mathbf{p}_{2f}\}$ with $\mathbf{p}_i = 2^{2f-i+\ell_m}$ for $i \in [2f+1]$

Appendix C. Security Proofs for IFSS Schemes

We prove security of our IFSS schemes for LRS and ReluARS. Specifically, in each case we describe AUX

and $\mathcal{S}_{\mathsf{AUX}}$ as well as two simulators: one ($\mathcal{S}.\mathsf{Gen}$) for simulating the key and one ($\mathcal{S}.\mathsf{Eval}$) for simulating the online phase. We use $\mathcal{S}^{<}_{n,\ell}$ to denote the simulator for DCF with n-bit index and ℓ-bit payload, as per Definition 1.

C.1. LRS

$\underline{\mathsf{AUX}(\mathsf{r}_{\mathsf{in}}, \tilde{\mathsf{r}}_{\mathsf{out}}, \Delta^{\mathsf{B}}, \Delta^{\mathsf{A}}) :}$

1) $\hat{w}, \hat{t} \leftarrow \{0,1\}, \mathsf{t}_0^{(w)}, \mathsf{t}_0^{(t)} \leftarrow \{0,1\}^{\kappa}$
2) $\mathsf{t}_1^{(w)} := \mathsf{t}_0^{(w)} \oplus \hat{w} \cdot \Delta^{\mathsf{B}}$
3) $\mathsf{t}_1^{(t)} := \mathsf{t}_0^{(t)} \oplus \hat{t} \cdot \Delta^{\mathsf{B}}$
4) Output $(\hat{w}, \hat{t}, \mathsf{t}_0^{(w)}, \mathsf{t}_0^{(t)})$ and $(\hat{w}, \hat{t}, \mathsf{t}_1^{(w)}, \mathsf{t}_1^{(t)})$

$\underline{\mathcal{S}^{\mathsf{LRS}}_{n,f}.\mathsf{AUX}(b, (\mathsf{r}_{\mathsf{in}}, \tilde{\mathsf{r}}_{\mathsf{out}}, \Delta^{\mathsf{B}}, \Delta^{\mathsf{A}}), (\hat{w}, \hat{t}, \mathsf{t}_b^{(w)}, \mathsf{t}_b^{(t)})) :}$

1) $\mathsf{t}_{1-b}^{(w)} := \mathsf{t}_b^{(w)} \oplus \hat{w} \cdot \Delta^{\mathsf{B}}$
2) $\mathsf{t}_{1-b}^{(t)} := \mathsf{t}_b^{(t)} \oplus \hat{t} \cdot \Delta^{\mathsf{B}}$
3) Output $\mathsf{aux}_{1-b} = (\hat{w}, \hat{t}, \mathsf{t}_{1-b}^{(w)}, \mathsf{t}_{1-b}^{(t)})$

$\underline{\mathcal{S}^{\mathsf{LRS}}_{n,f}.\mathsf{Gen}(b) :}$

1) Run the simulators for FSS schemes $\mathsf{FKOS\text{-}Gen}^{<}_n$ and $\mathsf{FKOS\text{-}Gen}^{<}_f$ to get keys $k^{<}_{n,b},\ k^{<}_{f,b}$
2) $\langle r^w \rangle_b, \langle r^t \rangle_b \leftarrow \{0,1\}^{1+\kappa}$, $[\![\mathbf{T}]\!]_b \leftarrow \mathbb{Z}^8_{\widetilde{N}}$
3) Output $k_b = (k^{<}_{n,b}, k^{<}_{f,b}, \langle r^{(w)} \rangle_b, \langle r^{(t)} \rangle_b, [\![\mathbf{T}]\!]_b)$

$\underline{\mathcal{S}^{\mathsf{LRS}}_{n,f}.\mathsf{Eval}(\hat{x}) :}$

1) Parse k_b as $(k^{<}_{n,b}, k^{<}_{f,b}, \langle r^{(w)} \rangle_b, \langle r^{(t)} \rangle_b, [\![\mathbf{T}]\!]_b)$
2) $\langle w \rangle_b \leftarrow \mathsf{FKOS\text{-}Eval}^{<}_n(k^{<}_{n,b}, \hat{x})$
3) $\langle t \rangle_b \leftarrow \mathsf{FKOS\text{-}Eval}^{<}_f(k^{<}_{f,b}, \hat{x} \bmod 2^f)$
4) $\langle \hat{w} \rangle_b = (\hat{w}_b, \mathsf{t}_b^{(w)}) := \langle w \rangle_b \oplus \langle r^{(w)} \rangle_b$
5) $\langle \hat{t} \rangle_b = (\hat{t}_b, \mathsf{t}_b^{(t)}) := \langle t \rangle_b \oplus \langle r^{(t)} \rangle_b$
6) Choose uniform $\hat{w}_{1-b}$ and $\hat{t}_{1-b}$ and send them to $\mathcal{A}$
7) $\hat{w} := \hat{w}_0 \oplus \hat{w}_1$, $\hat{t} := \hat{t}_0 \oplus \hat{t}_1$
8) $[\![\hat{y}]\!]_b := \mathsf{LRS}_{n,f}(\hat{x}) + [\![\mathbf{T}]\!]_b[\hat{w}, \hat{t}]$
9) Output $[\![\hat{y}]\!]_b$ and $\mathsf{aux}_b = (\hat{w}, \hat{t}, \mathsf{t}_b^{(w)}, \mathsf{t}_b^{(t)})$

The view of adversary in the real world contains the key k_b and reconstuction messages $\hat{w}_{1-b}, \hat{t}_{1-b}$. The key contains fresh shares and a DCF key. Indistinguishability of the above simulation follows from the security of DCF and fact that $\hat{w}_{1-b}, \hat{t}_{1-b}$ (and hence $\hat{w}, \hat{t}$) are uniform in both the real and simulated views.

C.2. Proof for ReluARS Scheme

$\underline{\mathsf{AUX}(\mathsf{r}_{\mathsf{in}}, \tilde{\mathsf{r}}_{\mathsf{out}}, \Delta^{\mathsf{B}}, \Delta^{\mathsf{A}}) :}$

1) $\hat{d}, \hat{w}, \hat{t} \leftarrow \{0,1\}, \mathsf{t}_b^{(d)}, \mathsf{t}_b^{(w)}, \mathsf{t}_b^{(t)} \leftarrow \{0,1\}^{\kappa}$
2) $\mathsf{t}_{1-b}^{(d)} := \mathsf{t}_b^{(d)} \oplus \hat{d} \cdot \Delta^{\mathsf{B}}$
3) $\mathsf{t}_{1-b}^{(w)} := \mathsf{t}_b^{(w)} \oplus \hat{w} \cdot \Delta^{\mathsf{B}}$
4) $\mathsf{t}_{1-b}^{(t)} := \mathsf{t}_b^{(t)} \oplus \hat{t} \cdot \Delta^{\mathsf{B}}$
5) Output $\mathsf{aux}_b = (\hat{d}, \hat{w}, \hat{t}, \mathsf{t}_b^{(d)}, \mathsf{t}_b^{(w)}, \mathsf{t}_b^{(t)})$ and $\mathsf{aux}_{1-b} = (\hat{d}, \hat{w}, \hat{t}, \mathsf{t}_{1-b}^{(d)}, \mathsf{t}_{1-b}^{(w)}, \mathsf{t}_{1-b}^{(t)})$

$\underline{\mathcal{S}^{\mathsf{LRS}}_{n,f}.\mathsf{AUX}(b, (\mathsf{r}_{\mathsf{in}}, \tilde{\mathsf{r}}_{\mathsf{out}}, \Delta^{\mathsf{B}}, \Delta^{\mathsf{A}}), (\hat{w}, \hat{t}, \mathsf{t}_b^{(w)}, \mathsf{t}_b^{(t)})) :}$

1) $\mathsf{t}_{1-b}^{(d)} := \mathsf{t}_b^{(d)} \oplus \hat{d} \cdot \Delta^{\mathsf{B}}$
2) $\mathsf{t}_{1-b}^{(w)} := \mathsf{t}_b^{(w)} \oplus \hat{w} \cdot \Delta^{\mathsf{B}}$
3) $\mathsf{t}_{1-b}^{(t)} := \mathsf{t}_b^{(t)} \oplus \hat{t} \cdot \Delta^{\mathsf{B}}$
4) Output $\mathsf{aux}_{1-b} = (\hat{d}, \hat{w}, \hat{t}, \mathsf{t}_{1-b}^{(d)}, \mathsf{t}_{1-b}^{(w)}, \mathsf{t}_{1-b}^{(t)})$

$\underline{\mathcal{S}^{\mathsf{ReluARS}}_{n,f}.\mathsf{Gen}(b) :}$

1) Run the simulators for FSS schemes $\mathsf{FKOS\text{-}Gen}^{<}_n$ and $\mathsf{FKOS\text{-}Gen}^{<}_f$ to get keys $k^{<}_{n,b},\ k^{<}_{f,b}$
2) $\langle r^d \rangle_b, \langle r^w \rangle_b, \langle r^t \rangle_b \leftarrow \{0,1\}^{1+\kappa}$
3) $[\![\mathbf{T}]\!]_b \leftarrow \mathbb{Z}^{16}_{\widetilde{N}}$, $[\![\mathbf{S}]\!]_b \leftarrow \mathbb{Z}^4_{\widetilde{N}}$
4) Output $k_b = (k^{<}_{n,b}, k^{<}_{f,b}, \langle \mathsf{r}^{(d)} \rangle_b, \langle \mathsf{r}^{(w)} \rangle_b, \langle \mathsf{r}^{(t)} \rangle_b, [\![\mathbf{T}]\!]_b, [\![\mathbf{S}]\!]_b)$

$\underline{\mathcal{S}^{\mathsf{ReluARS}}_{n,f}.\mathsf{Eval}(\hat{x}) :}$

1) Parse k_b as $(k^{<}_{n,b}, k^{<}_{f,b}, \langle \mathsf{r}^{(d)} \rangle_b, \langle \mathsf{r}^{(w)} \rangle_b, \langle \mathsf{r}^{(t)} \rangle_b, [\![\mathbf{T}]\!]_b, [\![\mathbf{S}]\!]_b)$
2) $\hat{y} := \hat{x} + 2^{n-1} \bmod 2^n$
3) $\langle w \rangle_b \leftarrow \mathsf{FKOS\text{-}Eval}^{<}_n(k^{<}_{n,b}, \hat{x})$
4) $\langle d \rangle_b \leftarrow \mathsf{FKOS\text{-}Eval}^{<}_n(k^{<}_{n,b}, \hat{y}) \oplus \mathsf{MSB}(\hat{y}) \oplus \langle w \rangle_b$
5) $\langle t \rangle_b \leftarrow \mathsf{FKOS\text{-}Eval}^{<}_f(k^{<}_{f,b}, \hat{x} \bmod 2^f)$
6) $\langle \hat{d} \rangle_b = (\hat{d}_b, \mathsf{t}_b^{(d)}) := \langle d \rangle_b \oplus \langle \mathsf{r}^{(d)} \rangle_b$
7) $\langle \hat{w} \rangle_b = (\hat{w}_b, \mathsf{t}_b^{(w)}) := \langle w \rangle_b \oplus \langle \mathsf{r}^{(w)} \rangle_b$
8) $\langle \hat{t} \rangle_b = (\hat{t}_b, \mathsf{t}_b^{(t)}) := \langle t \rangle_b \oplus \langle \mathsf{r}^{(t)} \rangle_b$
9) Choose uniform $\hat{d}_{1-b}, \hat{w}_{1-b}$ and $\hat{t}_{1-b}$ and send them to $\mathcal{A}$
10) $\hat{d} := \hat{d}_0 \oplus \hat{d}_1, \hat{w} := \hat{w}_0 \oplus \hat{w}_1$, $\hat{t} := \hat{t}_0 \oplus \hat{t}_1$
11) $[\![\hat{y}]\!]_b := [\![\mathbf{S}]\!]_b[\hat{d}] \cdot \mathsf{LRS}_{n,f}(\hat{x}) + [\![\mathbf{T}]\!]_b[\hat{d}, \hat{w}, \hat{t}]$
12) Output $[\![\hat{y}]\!]_b$ and $\mathsf{aux}_b = (\hat{d}, \hat{w}, \hat{t}, \mathsf{t}_b^{(d)}, \mathsf{t}_b^{(w)}, \mathsf{t}_b^{(t)})$

Appendix D.
Meta-Review

The following meta-review was prepared by the program committee for the 2025 IEEE Symposium on Security and Privacy (S&P) as part of the review process as detailed in the call for papers.

D.1. Summary

The paper presents SHARK, a maliciously secure two-party protocol for secure inference by extending function secret sharing. The proposed protocol is sound and efficient. The evaluation on PPML benchmark models and datasets demonstrates the practical performance of SHARK.

D.2. Scientific Contributions

- Provides a valuable step forward in an established field

D.3. Reasons for Acceptance

1) SHARK enables efficient secure inference under malicious security by realizing a set of new FSS-based protocols.
2) The evaluation over PPML tasks including transformers demonstrates high efficiency of SHARK.